

ConquerorLab

Écrire des règles, écrire une IA,
et faire parler les mesures

Cours complet — sujets de stage A1 et A2

Table des matières

0	Avant-propos	1
0.1	Pourquoi cet atelier existe	1
0.2	À qui s'adresse ce cours	1
0.3	Le kit, et sur quoi il tourne	1
0.3.1	Compiler pour une autre plateforme	2
0.3.2	Le kit évoluera	2
0.4	Lancer l'atelier	3
0.5	La première minute.	3
0.6	Trois gestes pour commencer	4
0.7	Ce que ce cours ne couvre pas	4
1	Le principe : trois dossiers, un contrat	5
1.1	Trois dossiers, trois natures de contribution	5
1.2	Le cycle de travail.	5
1.3	Le contrat : deux tables de pointeurs.	6
1.4	Qui parle à qui	6
1.5	L'état est opaque, la vue est en lecture seule.	7
1.6	Les quatre exigences non négociables	8
1.7	Ce qu'un module a le droit d'inclure	9
1.8	Quand le contrat change : majeure et mineure.	10
1.9	Afficher un état pour comprendre — le panneau Sortie	10
1.10	La géométrie fournie, en trois fonctions	11
2	Écrire des règles en trois fonctions	13
2.1	Le compte, mesuré	13
2.2	Le module entier	13
2.3	L'essayer	14
2.4	Ce que l'échafaudage vous coûte	14
2.5	Quand partir — et ce n'est pas un échec	15
3	Écrire une IA.	16
3.1	Pourquoi commencer par une IA nulle	16
3.2	Le point de conception central	16
3.3	Les neuf fonctions	17
3.4	ChooseMove, ligne à ligne	17
3.4.1	Le compte-rendu	17
3.5	Le PRNG : porté par l'instance	18
3.6	Le thread, et le budget	19
3.6.1	Respecter le budget	19
3.7	Les paliers de difficulté	20
3.8	Évaluer une position sans connaître les règles	20
3.9	OnMovePlayed : ce qui distingue un MCTS d'une recherche jetable.	20
3.10	Votre première IA, en quatre gestes	21
4	Fabriquer une grille en JSON	22
4.1	Pourquoi le plateau est une donnée	22
4.2	Où déposer le fichier	22
4.3	Le format, sur un exemple minuscule.	22
4.4	Hexagone : la conversion qui piège	23
4.5	La symétrie n'est pas une coquetterie	24
4.6	Quand le moteur refuse votre fichier.	24

5 Définir sa grille en C++	27
5.1 Le malentendu : «je suis obligé de passer par du JSON»	27
5.2 Ce qui vous échappait vraiment : la projection écran.	27
5.3 Un plateau circulaire, entièrement en C++.	28
5.3.1 Le voisinage, défini par nous	29
5.3.2 Où dessiner : GetCellCenter	29
5.3.3 Quelle forme : GetCellShape	30
5.3.4 Le déclarer aux autres : GetNeighbors	30
5.4 Comment l'atelier s'y adapte	31
5.5 JSON ou C++ : lequel choisir.	32
5.6 État vérifié	32
6 Se servir de l'atelier.	34
6.1 Les sept panneaux	34
6.2 Lire le plateau	34
6.2.1 La barre d'état, et les boutons	34
6.3 Le panneau Sortie : vos propres traces	35
6.4 Reproduire un bug : le journal	35
6.5 Deux garde-fous contre les erreurs silencieuses	36
6.5.1 L'incohérence voisinage / coups légaux	36
6.5.2 La signature de configuration	36
6.6 La campagne : ce qu'elle mesure, et comment.	37
6.6.1 L'inversion des côtés	37
6.6.2 Les tuiles rouges	37
6.6.3 Les deux histogrammes	38
6.7 Une méthode de mesure qui tient	38
6.8 Quand ça ne marche pas.	39
6.9 Le banc d'essai, hors interface	39
7 Exemples complets — l'échelle, barreau par barreau.	41
7.1 Les trois échelles	41
7.2 D'abord : la couche confortable	41
7.2.1 Ce qu'elle apporte	42
7.2.2 La preuve : la même IA, deux fois	42
7.3 Une IA, en trois temps	43
7.3.1 Temps 1 — tirer au hasard	43
7.3.2 Temps 2 — regarder un coup	43
7.3.3 Temps 3 — regarder loin	44
7.3.4 Ce que ça donne, mesuré	45
7.4 Des règles : trois fonctions, ou vingt et une	46
7.4.1 Pourquoi les dix-huit autres deviennent gratuites	46
7.4.2 Ce que le banc d'essai en dit	47
7.5 Des règles, sans échafaudage : RegleContratNu.cpp	47
7.6 Un plateau : les deux voies	48
7.6.1 En JSON, le plus petit possible	48
7.6.2 En C++, quand la forme est une formule	48
7.7 Le chemin conseillé.	49

8	Quand l'échafaudage ne suffit plus : le contrat nu	51
8.1	Ce que ce moteur fait	51
8.2	Les vingt-trois fonctions, par famille	51
8.3	La mémoire : jamais de new ni de delete bruts.	52
8.4	Les paramètres : aucune constante en dur	52
8.4.1	Borner, ne pas refuser	53
8.5	L'état : votre représentation, leur vue.	54
8.6	Générer les coups	54
8.6.1	Le cas de PASSER	55
8.7	L'équivalence qui teste tout le reste	56
8.8	Appliquer un coup, et la cascade.	56
8.8.1	Les événements : décrire, pas animer	57
8.9	Fin de partie et garde-fou	57
8.10	Les deux symboles exportés	58
8.11	Votre premier module, en quatre gestes	58

Avant-propos

0.1 Pourquoi cet atelier existe

Les règles de Conqueror ne sont pas connues. Plusieurs sont ouvertement suspectes, et aucune ne se tranche par le débat. La version précédente du document de règles s'annonçait «**TOTALEMENT VERROUILLÉE**» avec «**16 décisions prises**»; plusieurs de ces décisions étaient mesurablement mauvaises, et personne ne pouvait le savoir sans les simuler.

D'où la règle de travail du projet, qui commande tout le reste.

REGLES_COMPLETES_v2.md:31-35

```
1 La regle de travail de ce projet. Toute valeur numerique de ce document est
2 un PARAMETRE NOMME, jamais une constante. Le moteur de regles doit exposer
3 ces parametres pour qu'ils soient modifiables SANS RECOMPILATION. Une regle
4 n'est pas defendue par argument : elle est refutee ou confirmee par
5 simulation de masse.
```

ConquerorLab est l'instrument qui produit cette simulation de masse. Il permet de **tester une règle en dix minutes au lieu de trois semaines**.

0.2 À qui s'adresse ce cours

Vous êtes...	Lisez au minimum
A1 — vous écrivez le moteur de règles	chapitres 0, 1, 2, 4, 5, 6
A2 — vous écrivez l'IA adverse	chapitres 0, 1, 3, 6
designer — vous réglez et vous mesurez	chapitres 0, 1, 4, 6
vous reprenez le projet	tout, dans l'ordre

Prérequis : savoir écrire du C++ — pas du C++ avancé. Les trois exemples de ce cours n'utilisent ni templates, ni héritage, ni exceptions. Aucune connaissance de Nkntseu n'est nécessaire pour les chapitres 2, 3 et 5.

0.3 Le kit, et sur quoi il tourne

Vous avez reçu un dossier — ou une archive ConquerorLab-Kit.zip — qui contient tout : l'atelier, les en-têtes, les bibliothèques, les exemples, et ce cours en PDF. **Il n'y a rien à installer sauf un compilateur** (voir LISEZMOI.txt).

Disposition du kit

```
1 ConquerorLab.exe      1'atelier
2 include/              TOUS les en-tetes, a plat
```

```

3      Conqueror/          le contrat
4      NKCore/ NKMath/ NKContainers/ NKLogger/ ...
5  lib/                    les bibliotheques liees a vos modules
6  travail/rules/          <- vos moteurs de regles      (.cpp)
7  travail/ai/             <- vos IA                      (.cpp)
8  travail/boards/         <- vos grilles                (.json)
9  exemples/               trois modules complets, a recopier

```

include/ reflète exactement ce que vous tapez

include/NKMath/NkFunctions.h correspond à #include "NKMath/NkFunctions.h". La première version du kit recopiait l'arborescence du dépôt — [Kernel/Foundation/NKCore/src/NKCore/...](#) Foundation, System, src sont de la plomberie de dépôt : trois niveaux qui ne veulent rien dire quand on écrit des règles de jeu, et qui séparent les modules de ce qu'ils incluent. L'atelier n'a donc qu'un seul -I.

Windows uniquement, pour l'instant

Le kit livré est **Windows x64**. Ce n'est pas une limite de conception : l'atelier est écrit sur NKGui/NKCanvas, dont les portages Linux, Android et Web existent. C'est une limite de **ce qui a été construit et vérifié à ce jour** — et livrer un binaire non testé serait pire que ne rien livrer.

Si vous travaillez sous Linux ou macOS, vous pouvez cloner le dépôt et compiler vous-même. Dites-le : c'est le genre de retour qui fait avancer la liste.

0.3.1 Compiler pour une autre plateforme

Le dépôt se compile avec jenga, l'outil de construction de la maison :

Compilation depuis les sources

```

1  git clone <depot Nkentsau>
2  cd Nkentsau
3  jenga build --target ConquerorLab --config Release

```

Le binaire sort dans Build/Bin/Release-<Plateforme>/ConquerorLab/.

Deux choses à savoir avant de vous lancer :

- **Le compilateur de modules est PC seulement.** Sur Android et Web il n'y a pas de compilateur sur l'appareil : seuls les modules compilés *dans* l'application sont disponibles. L'atelier reste jouable et mesurable ; c'est le rechargement à chaud qui disparaît. Limite réelle et assumée.
- **Les chemins des bibliothèques changent.** LibDir() vise Build/Lib/<Config>-<Plateforme>, et l'extension des modules passe de .dll à .so (.dylib sur macOS). Tout cela est déjà écrit et compilé conditionnellement — mais jamais **exécuté** ailleurs que sur Windows. Attendez-vous à corriger deux ou trois choses, et signalez-les.

0.3.2 Le kit évoluera

Ce n'est pas une livraison figée. Les retours — un message d'erreur incompréhensible, un panneau qui déborde, une fonction du contrat qui manque — donnent lieu à une nouvelle version du kit et du cours.

Ce qu'il faut retenir

Les deux nouveautés les plus récentes sont nées exactement comme ça : le panneau *Sortie* et l'ouverture de la géométrie au C++ (chapitre 5) viennent tous deux d'une remarque, pas d'un plan.

Le fichier `VERSION.txt` du kit dit quelle version vous avez. Citez-la dans vos retours.

0.4 Lancer l'atelier

Applications/ConquerorLab/README.md — section « Lancer »

```
1 cd D:\Projets\2026\Nkentseu\Nkentseu
2 jenga build --target ConquerorLab --config Release
3 .\Build\Bin\Release-Windows\ConquerorLab\ConquerorLab.exe
```

Trois dossiers sont créés au premier lancement :

Les trois dossiers de contribution

```
1 depot : Build/ConquerorLab/rules|ai|boards
2 kit    : travail/rules|ai|boards
```

Le chemin diffère parce que « Build » est un mot de dépôt : dans un kit, personne ne construit rien. C'est `NkcLayout.h` qui détecte la disposition, et lui seul les connaît toutes les deux.

Le troisième contient déjà `exemple\_plateau\_par\_default.json`, écrit par l'atelier : c'est le plateau du moteur de référence, exporté. Un format qu'on peut lire vaut mieux qu'un format qu'on décrit.

0.5 La première minute

À l'ouverture, rien ne bouge, et c'est voulu :

- le **joueur 0 est humain**, les autres sont pilotés par l'IA de référence ;
- la partie est **en pause** : c'est vous qui la lancez.

La barre du plateau dit ce que l'atelier attend. Elle affiche à tout instant l'une de ces phrases :

Ce que vous lisez	Ce que ça veut dire
au tour du joueur humain	à vous : cliquez un totem, puis une case verte
en pause — Lecture ou Pas à pas l'IA réfléchit	c'est à une IA de jouer, mais rien ne tourne un thread calcule ; l'interface reste vivante
rejeu en pause	vous relisez le passé, le jeu attend
partie terminée	le décompte est affiché au centre
IA introuvable	un vrai problème : allez au panneau Modules

Ce qu'il faut retenir

Une interface immobile et muette se lit comme une panne. C'est exactement ce qui s'est passé pendant le développement : le premier réglage mettait le joueur 0 en humain, l'atelier attendait un clic, et personne ne comprenait pourquoi « la simulation ne marchait pas ». La barre d'état a été ajoutée pour cela. **Si l'atelier semble bloqué, lisez-la avant de chercher un bug.**

0.6 Trois gestes pour commencer

1. **Jouer un coup.** Cliquez un de vos totems. Les destinations légales s'entourent de vert. Survolez-en une : les totems ennemis qu'elle *retournerait* s'entourent de rouge. C'est la lecture tactique centrale du jeu — « quelle surface de contact suis-je en train d'offrir? ». Cliquez pour jouer.
2. **Laisser tourner.** Bouton **Lecture**. L'IA répond. Le dernier coup est marqué d'un anneau orange qui pulse une seconde.
3. **Passer en mode mesure.** Bouton **IA vs IA** : tous les sièges passent à l'IA et la partie tourne seule. C'est la configuration dans laquelle l'atelier répond à une question.

0.7 Ce que ce cours ne couvre pas

- **Le jeu lui-même.** Les règles de Conqueror sont dans `REGLES\_COMPLETES\_v2.md`. Ce cours explique l'outil qui les teste.
- **NKGui, NKCanvas, le moteur.** Un module de stagiaire n'y touche pas. Pour modifier l'*interface* de l'atelier, c'est le cours *NkCanvas & NKGui* qu'il vous faut.
- **Les paliers 1 et 2** (fusion, ressources, pouvoirs, artefacts). Le moteur de référence livré s'arrête au palier 0. Le contrat, lui, les prévoit déjà : c'est pour cela que `NkcMove` porte des champs `fuseCells` et `powerId` qui restent à zéro aujourd'hui.

Le principe : trois dossiers, un contrat

Ce chapitre est la carte. Tout le reste en découle, et si vous ne deviez lire qu'un chapitre, ce serait celui-ci.

1.1 Trois dossiers, trois natures de contribution

Vous ne manipulez **jamais** de DLL et ne lancez **jamais** de commande de build. Vous déposez un fichier dans un dossier, et l'atelier fait le reste.

J'ajoute...	Je dépose...	Ce qui se passe
un moteur de règles	rules/mes_regles.cpp	déecté → compilé → chargé → panneau <i>Modules</i>
une IA	ai/mon_ia.cpp	idem → liste des pilotes, panneau <i>Joueurs</i>
une grille	boards/diamant.json	aucune compilation → panneau <i>Règles</i>

La différence entre les deux premiers et le troisième n'est pas une commodité, c'est une décision de conception.

Une règle est du code, un plateau est une donnée

Le plateau n'est pas une constante du code : c'est un descripteur sérialisable, éditable à la souris dans le moteur de test. REGLES_COMPLETES_V2.MD :95

Une règle décrit un *comportement* : elle se programme. Un plateau décrit une *forme* : il se décrit. Confondre les deux vous condamne à recompiler pour essayer un hexagone au lieu d'un carré — et donc à ne jamais l'essayer.

Le chapitre 5 montre la troisième voie : un plateau décrit **en C++**, sans JSON, quand sa forme ne se laisse pas mettre en tableau.

1.2 Le cycle de travail

Ce qui se passe quand vous sauvegardez

```

1 1. vous ecrivez   Build/ConquerorLab/rules/mes_regles.cpp
2 2. vous sauvegardez
3 3. l'atelier detecte le changement (il scrute une fois par seconde)
4 4. il COMPILE -> il charge -> votre module apparait dans le menu
5 5. vous rejouez une partie, sans avoir quitte l'application
```

En cas d'erreur, **la sortie complète du compilateur** s'affiche dans le panneau *Modules*. C'est le seul retour que vous aurez : pas de terminal, pas de chaîne de build à apprendre.

La compilation fige l'atelier une à deux secondes

C'est assumé, et documenté dans [src/ConquerorLab/NkcSession.h](#) :

La compilation est SYNCHRONE : quand le stagiaire sauvegarde, l'atelier se fige une seconde ou deux, puis son module apparaît. C'est assumé — un compilateur piloté depuis un thread annexe demanderait une file de travaux et un verrou sur le catalogue, pour gagner deux secondes toutes les cinq minutes.

Si l'atelier se fige au moment où vous sauvegardez, il ne plante pas : il vous compile.

1.3 Le contrat : deux tables de pointeurs

Un module ne dérive d'aucune classe et n'exporte aucun symbole C++. Il remplit une **structure de pointeurs de fonction** et l'expose par deux fonctions C.

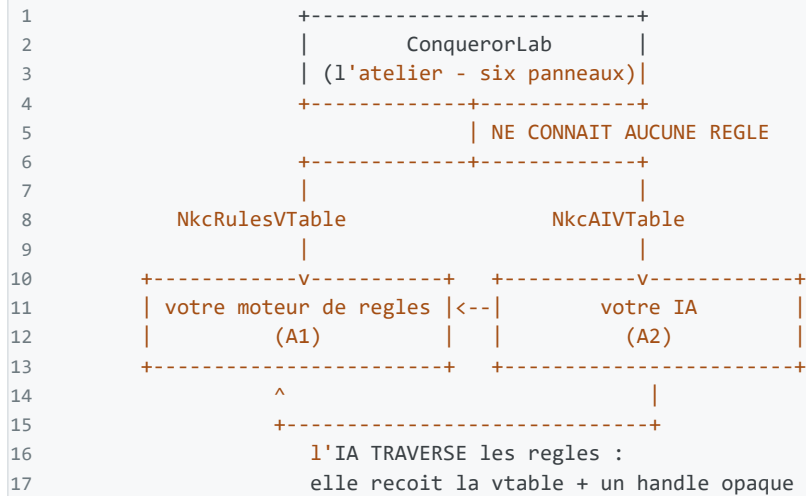
include/Conqueror/ConquerorRulesABI.h:255-273 (extrait)

```
1 struct NkcRulesFactory {
2     NkcRulesInfo info;
3     NkcRulesVTable vtable;
4 };
5
6 // Les DEUX symboles que tout module de regles doit exporter.
7 using NkcRulesGetFactoryFn = void (*)(NkcRulesFactory *out);
8 using NkcRulesSetAllocatorFn = void (*)(NkcAllocFn a, NkcFreeFn f);
9
10 #define NKC_RULES_SYM_GET_FACTORY "nkc_rules_get_factory"
11 #define NKC_RULES_SYM_SET_ALLOC "nkc_rules_set_allocator"
```

Pourquoi une table de pointeurs plutôt qu'une classe abstraite ? Parce qu'une classe abstraite impose une **ABI C++** — disposition de vtable, décoration des noms, modèle d'exception — qui change d'un compilateur à l'autre et parfois d'une version à l'autre. Une structure de pointeurs de fonction, non.

1.4 Qui parle à qui

Le schema qui explique le decoupage des deux stages



Trois propriétés découlent de ce schéma, et ce sont elles qui font tenir le stage :

1. **A2 démarre en semaine 1**, sur un module bouchon, sans attendre A1;
2. **quand A1 change son code interne, l'IA continue de fonctionner** — elle n'a jamais vu ce code;
3. **les deux ne peuvent pas diverger sur les règles** : il n'en existe qu'une implémentation, et l'IA la traverse.

1.5 L'état est opaque, la vue est en lecture seule

Vous choisissez librement la représentation interne de votre partie. Ni l'atelier ni l'IA ne la voient. Ce qu'ils voient est une **vue**.

include/Conqueror/ConquerorRulesABI.h:92-109 (abrege)

```

1 struct NkcStateView {
2     const NkcCellView *cells      = nullptr; // cellCount entrees
3     const NkcCoord *coords      = nullptr; // meme indexation
4     uint32 cellCount = 0;
5
6     NkcTopology topology      = NkcTopology::HexPointy;
7     uint8 playerCount = 2;
8     uint8 current      = 0; // joueur au trait
9     uint8 finished     = 0;
10    int8 winner        = -2; // -1 = nul, -2 = en cours
11    uint32 turn        = 0;
12
13    const int32 *energy      = nullptr;
14    const int32 *conquestTenths = nullptr; // en DIXIEMES
15    const int32 *totemCount  = nullptr;
16 };

```

Les pointeurs de la vue appartiennent au module

Ils restent valides **jusqu'au prochain appel mutant** sur cet état (ApplyMove, Setup, DeserializeState, CloneState). L'appelant ne libère jamais rien.

En pratique : après avoir appliqué un coup, **redemandez la vue**. Les champs scalaires (current, turn, finished) sont des *copies* faites au moment de GetView — ils ne se mettent pas à jour tout seuls.

1.6 Les quatre exigences non négociables

Elles ne sont pas des préférences de style. Elles conditionnent l'existence même du sujet A2 et de toute campagne de mesure.

REGLES_COMPLETES_v2.md:583-590

```
1 1. Zero flottant dans la logique de regles. Entiers partout, PC en dixiemes.  
2 2. PRNG porte par l'etat de partie, serialise avec lui. Jamais de generateur  
3   global.  
4 3. Rejeu bit-a-bit : seed + liste de coups -> etat final identique, sur  
5   Windows et sur Linux, quel que soit le compilateur.  
6 4. Aucun ordre d'iteration de conteneur ne doit etre observable dans le  
7   resultat. Toute resolution ambigue est arbitree par un critere explicite  
8   (index de joueur, coordonnee), jamais par l'ordre de parcours d'une table.
```

Chacune a une raison très concrète.

1 — Zéro flottant

Les Points de Conquête valent $1 + \text{niveau} \times 0,5 + \text{ennemis} \times 0,2 + \text{alliés} \times 0,1$. En virgule flottante, **l'ordre d'accumulation change le dernier bit** selon le compilateur et la plateforme, et le rejeu bit-à-bit tombe. Le moteur stocke donc les PC en **dixièmes entiers** : base 10, niveau $\times 5$, ennemi $\times 2$, allié $\times 1$. La division par 10 est un fait d'*affichage* — elle n'existe nulle part dans la logique.

2 — PRNG porté par l'état

L'IA clone une position des milliers de fois par seconde. Avec un générateur global, deux clones piochent dans le même flux et le rejeu meurt. Au palier 0 il n'y a aucun aléatoire dans Conqueror; le champ existe quand même, parce que l'ajouter plus tard obligerait à re-sérialiser tous les états déjà enregistrés.

3 — Rejeu bit-à-bit

C'est ce qui permet de coller un journal dans un rapport de bug et de le rejouer à l'identique ailleurs. Le panneau *Journal* affiche l'empreinte après **chaque** coup : quand deux rejeux divergent, elle dit *où*, pas seulement *qu'ils diffèrent*.

4 — Aucun ordre observable

Si votre générateur de coups parcourt une table de hachage, l'ordre des coups change entre deux exécutions. L'IA choisit alors un coup différent, la partie diverge, et vos dix mille parties ne mesurent plus rien.

Ce qu'il faut retenir

`GenerateLegalMoves` **doit** produire les coups dans un ordre stable et reproductible. Dans les exemples de ce cours : cases par index croissant, puis voisins dans l'ordre déclaré. Ne le changez jamais sans raison — et si vous le changez, sachez que toutes les mesures antérieures deviennent incomparables.

1.7 Ce qu'un module a le droit d'inclure

La frontière est `NKCanvas` / `NKGui` : tout ce qui est en dessous vous est ouvert, rien de ce qui est au-dessus ne l'est.

`src/ConquerorLab/NkcModuleCompiler.h` — `StackIncludes`

```
1  /// L'atelier ne donnait au depart que trois -I et aucun lien : un
2  /// module compilait sans rien lier du moteur. C'etait elegant, et
3  /// trop etroit -- un moteur de regles a besoin d'une chaine, d'un
4  /// tableau dynamique, d'un formatage, d'un journal. Reecrire tout
5  /// cela a la main dans chaque module est du temps vole au jeu.
```

Vous avez donc droit à :

Ce que vous incluez	Ce que ça vous donne
	le contrat
<code>Conqueror/ConquerorRulesABI.h</code>	
<code>Conqueror/ConquerorGeometry.h</code>	voisinage, distance — inline et entiers
<code>NKCore/NkTypes.h</code>	<code>uint32</code> , <code>int8</code> , <code>usize</code>
<code>NKContainers/String/NkString.h</code>	<code>NkString</code> , <code>NkFormat</code>
<code>NKContainers/Sequential/NkVector.h</code>	<code>NkVector</code> , <code>NkList</code>
<code>NKContainers/Associative/NkMap.h</code>	<code>NkMap</code> , <code>NkSet</code>
<code>NKMath/NkFunctions.h</code>	<code>math::NkSqrt</code> , <code>NkClamp...</code>
<code>NKLogger/NkLog.h</code>	<code>logger.Infof(...)</code>
<code>NKFileSystem/NkFile.h</code>	<code>NkFile</code> , <code>NkDirectory</code>
<code>NKThreading/NkThread.h</code>	<code>NkThread</code> , <code>NkMutex</code>
la bibliothèque standard C	<code><cstring></code> , <code><cstdio></code> ...

Vous n'avez **pas** droit à `NKCanvas`, `NKGui`, `NKWindow`, `NKRenderer` — un moteur de règles ne dessine pas. C'est l'atelier qui affiche, à partir de la vue que vous exposez. **Cette frontière est le cœur du contrat**, pas une restriction administrative : c'est elle qui permet de changer entièrement l'affichage sans toucher une règle, et inversement.

1.8 Quand le contrat change : majeure et mineure

Le contrat va évoluer pendant vos huit semaines. La question qui compte est : **faut-il tout recompiler à chaque fois ?**

include/Conqueror/ConquerorRulesABI.h

```
1 inline constexpr uint32 kRulesAbiMajor = 3;
2 inline constexpr uint32 kRulesAbiMinor = 1;
```

- **MAJEURE** — un champ change de type ou de place, une signature change. Votre module est refusé : il faut recompiler, et le panneau *Modules* le dit.
- **MINEURE** — une fonction est ajoutée à la fin de la vtable. Votre module **continue de tourner**. L'atelier l'accepte et note : « module en ABI 3.0, atelier en 3.1 — il fonctionne, les fonctions ajoutées depuis ne sont pas disponibles. »

Pourquoi cette distinction existe : sans elle, le jour où j'ai ajouté trois entrées à la vtable, **tous** les modules devenaient « ABI 2, attendue 3 » — refusés alors qu'ils tournaient parfaitement. Vous faire recompiler tout votre travail parce que j'ai ajouté une fonction est exactement la friction qu'un contrat doit supprimer.

Ce que ça a coûté d'apprendre

La règle « on ajoute à la fin » est plus étroite qu'elle n'en a l'air : on ajoute à la fin de la **dernière structure de la chaîne**.

NkcRulesInfo précède NkcRulesVTable dans la fabrique. Grossir info **décale** la vtable : un module d'époque écrit sa table à l'ancien décalage, l'atelier la lit au nouveau, et on appelle des adresses au hasard. Premier essai : **segfault**. Les champs sont donc à la fin de NkcRulesFactory, après la vtable.

tests/NkcAbiCompat.cpp monte désormais la garde : il compile un module contre des en-têtes **figés** et le fait jouer avec l'atelier d'aujourd'hui. **11 vérifications, 0 échec**.

1.9 Afficher un état pour comprendre — le panneau Sortie

Votre module a sa **propre copie de la pile** : l'édition de liens est statique. Conséquence directe, et surprenante la première fois : un `logger.Infoof()` depuis votre code n'atteint pas la console de l'atelier. Vous écrivez dans *votre* journal, pas dans le sien.

Ce n'est pas un oubli — c'est la conséquence de l'isolation qui fait tout l'intérêt du système. Mais déboguer un moteur de règles sans pouvoir afficher un état, c'est déboguer à l'aveugle. L'atelier **injecte donc un puits** dans chaque module au chargement, et le branchement tient en une ligne :

exemples/rules/RegleContratNu.cpp — fin de fichier

```
1 NKC_MODULE_EXPORT void nkc_rules_set_allocator(NkcAllocFn a, NkcFreeFn f) {
2     gAlloc = a; gFree = f;
3 }
4 NKC_MODULE_LOGGING(rules) // <- une ligne, et NKC_LOG_* va dans « Sortie »
```

`NKC_MODULE_LOGGING(rules)` dans un module de règles, `NKC_MODULE_LOGGING(ai)` dans une IA. Il faut inclure `Conqueror/ConquerorLog.h`. À partir de là :

```
1 NKC_LOG_INFO("nouvelle partie : %d cases, %d joueurs, graine %llu",
2             kCells, (int32)s->playerCount, (unsigned long long)seed);
```

s'affiche dans le panneau **Sortie**, avec le nom de votre module et le niveau.

logger.Info marche aussi

Si vous incluez NKLogger, la macro branche en plus un NkISink sur le logger *privé* de votre module, qui repousse tout vers l'atelier. Le réflexe de quelqu'un qui connaît déjà Nkntseu fonctionne donc sans qu'il ait à l'apprendre.

Une seule contrainte : incluez `ConquerorLog.h` **après** NKLogger — la détection se fait sur son garde d'inclusion.

Deux limites à connaître

Sans atelier, tout part sur stderr. Banc d'essai, test en ligne de commande : personne n'injecte de puits, et vos traces sortent sous la forme [INFO] module: ... Elles ne disparaissent jamais en silence.

Ne journalisez pas dans une boucle chaude. NKC_LOG_* formate *avant* de savoir si quelqu'un écoute. Dans un rollout MCTS c'est un désastre — et le tampon de l'atelier est borné à 4096 lignes, donc vous perdriez de toute façon ce que vous cherchez. Une trace par coup, pas par nœud. Pour mesurer, il y a NkcAIResult et GetDebugJson.

L'isolation explique aussi `nkc_rules_set_allocator` : sans injection, les deux côtés allouent dans des tas distincts.

Les trois exemples de ce cours n'utilisent volontairement que `<cstring>` et `<cstdio>` : ils doivent rester lisibles par quelqu'un qui ne connaît pas encore Nkntseu. Rien ne vous oblige à cette austérité.

Pourquoi la projection écran n'est pas dans la géométrie

Aucune dépendance à NKMath : tout y est inline et entier, donc un module compilé par l'atelier n'a AUCUN symbole à lier. La projection écran (cos/sin/arrondi) vit ailleurs — c'est de la présentation, pas de la règle. CONQUERORGEOMETRY.H:10-13

NkRound de NKMath est un symbole *lié*. L'inclure dans le contrat ferait échouer l'édition de liens de **tous** les modules.

1.10 La géométrie fournie, en trois fonctions

C'est le socle commun. Le chapitre 5 montre comment s'en passer entièrement.

include/Conqueror/ConquerorGeometry.h:26-88 (signatures)

```
1 int32 NeighborCount(NkcTopology t) noexcept;
2 NkcCoord Neighbor(NkcTopology t, NkcCoord c, int32 i) noexcept;
3 int32 Distance(NkcTopology t, NkcCoord a, NkcCoord b) noexcept;
```

Quatre topologies, et le voisinage est une propriété de la topologie — jamais codé en dur ailleurs :

Topologie	Voisins	Coordonnées
HexPointy	6	axiales (q, r) , pointes haut/bas
HexFlat	6	axiales (q, r) , pointes gauche/droite
Square4	4	(x, y) , orthogonaux
Square8	8	(x, y) , + diagonales

Un seul type de coordonnée pour les quatre : c'est la topologie qui décide de l'interprétation.

L'ordre des voisins est **NORMATIF**

i-ème voisin de c. L'ORDRE EST NORMATIF : il fixe l'ordre de génération des coups, donc la reproductibilité. Ne jamais le changer. CONQUERORGEOMETRY.H :36-37

1 — Lire le contrat

Ouvrez [ConquerorRulesABI.h](#) et faites la liste des fonctions de `NkcRulesVTable` **sans regarder le chapitre 2**. Classez-les en quatre familles : cycle de vie, réglages, jeu, sérialisation. Combien y en a-t-il ? Laquelle vous paraît la plus difficile à implémenter, et pourquoi ?

2 — La cascade

Lisez [REGLES_COMPLETES_v2.md](#) §7.3. Une duplication retourne les ennemis voisins de la case où l'on vient de poser. Si un totem retourné a lui-même des voisins ennemis, sont-ils retournés à leur tour ? Trouvez la phrase qui tranche, et expliquez pourquoi le document dit que c'est « le point le plus facile à implémenter de travers ».

3 — Le flottant qui tue

Écrivez un petit programme qui additionne $0.1f$ cent fois, puis fait la même chose en dixièmes entiers, et compare au résultat exact 10.0 . Quel écart obtenez-vous ? Multipliez-le mentalement par dix mille parties et deux plateformes : vous tenez la raison de l'exigence n° 1.

Écrire des règles en trois fonctions

Un moteur de règles complet tient en trois méthodes.

Méthode	La question à laquelle elle répond
Construire	à quoi la partie ressemble au départ
CoupsPossibles	ce qu'on a le droit de faire
Appliquer	ce qui se passe quand on le fait

Tout le reste — créer et détruire une instance, cloner un état, le sérialiser, le hacher, dire si la partie est finie, dire qui a gagné, vérifier qu'un coup est légal — est **le même pour tout le monde**, et il est déjà écrit.

Le fichier de ce chapitre existe, il compile, et l'atelier le charge : `Applications/ConquerorLab/exemples/rules/RegleFacile.cpp`.

2.1 Le compte, mesuré

NkcRulesVTable compte **24 entrées**. Voici où elles vont.

ou vont les 24 entrees de la vtable

```
1 24 entrees de la vtable
2 3 contiennent VOTRE jeu    Construire, CoupsPossibles, Appliquer
3 18 sont ecrites une fois pour toutes dans ConquerorRegleFacile.h
4 3 sont optionnelles (geometrie) et restent nulles
```

Les recopier à chaque module, ce serait dix-huit occasions de se tromper sur du code qui n'a aucun rapport avec le jeu qu'on essaie de concevoir.

2.2 Le module entier

Voici un moteur qui joue vraiment. Rien n'a été coupé.

exemples/rules/RegleFacile.cpp — le module complet

```
1 #include "Conqueror/ConquerorRegleFacile.h"
2
3 using namespace nkentseu;
4 using namespace nkentseu::conqueror;
5 using namespace nkentseu::conqueror::facile;
6
7 struct MesRegles {
8
9     static constexpr int32 kCote = 5;
10
11     // 1. A quoi la partie ressemble au depart.
12     void Construire(Grille &g) {
```

```

13     g.topologie = NkcTopology::Square4;
14     g.nbJoueurs = 2;
15     g.AjouterRectangle(kCote, kCote);
16     g.PoserAuxCoins(2);
17 }
18
19 // 2. Ce qu'on a le droit de faire : dupliquer vers une case voisine VIDE.
20 void CoupsPossibles(const Partie &p, ListeCoups &out) {
21     NkcCoord voisins[8];
22     for (int32 i = 0; i < p.nbCases; ++i) {
23         if (p.cases[i].owner != static_cast<int8>(p.joueur)) continue;
24         const NkcCoord de = p.ou[i];
25         const int32 n = p.Voisins(de, voisins, 8);
26         for (int32 k = 0; k < n; ++k)
27             if (p.Vide(voisins[k]))
28                 out.Dupliquer(p.joueur, de, voisins[k]);
29     }
30     // PASSER est ajoute tout seul si cette liste reste vide.
31 }
32
33 // 3. Ce qui se passe quand on le fait.
34 void Appliquer(Partie &p, const NkcMove &m, Evenements &ev) {
35     p.Poser(m.to, m.player);
36     ev.Duplique(m.player, m.from, m.to);
37     p.conquete[m.player] += 10; // en DIXIEMES : 10 == 1,0 point
38
39     NkcCoord voisins[8];
40     const int32 n = p.Voisins(m.to, voisins, 8);
41     for (int32 k = 0; k < n; ++k) {
42         if (!p.Ennemie(voisins[k], m.player)) continue;
43         const int8 ancien = p.Proprietaire(voisins[k]);
44         p.Poser(voisins[k], m.player);
45         ev.Retourne(m.player, voisins[k], ancien);
46     }
47     p.PasserLaMain();
48 }
49 };
50
51 NKC_REGLES(MesRegles, "MesRegles", "1.0.0", "Moi")

```

La dernière ligne fabrique les vingt et une entrées de la vtable et les deux symboles exportés. C'est tout le module.

2.3 L'essayer

1. copiez exemples/rules/RegleFacile.cpp vers travail/rules/mes_regles.cpp;
2. changez le nom dans NKC_REGLES — sinon deux entrées identiques au menu;
3. sauvegardez, attendez une seconde;
4. panneau **Modules** → votre module apparaît → **Nouvelle partie**.

Si la compilation échoue, la sortie complète du compilateur s'affiche dans le panneau **Modules**. C'est votre seul retour : lisez-la.

2.4 Ce que l'échafaudage vous coûte

Il n'y a pas de magie, et la contrepartie se dit en une phrase : **Partie est une structure fixe, plate, sans pointeur.**

C'est ce qui permet aux dix-huit fonctions d'être justes **par construction** et non par relecture.

ce que le cadre doit faire	comment, parce que <code>Partie</code> est plate
cloner un état	une affectation
sérialiser	un <code>memcpy</code>
hacher	FNV-1A sur les octets
dire si un coup est légal	énumérer les coups possibles et comparer

Le prix : vous ne pouvez pas ranger dans `Partie` un état de taille libre — une liste qui grandit, un cache, un arbre. Tant que votre jeu tient dans les cases, les joueurs, l'énergie et la conquête, vous n'y penserez jamais.

2.5 Quand partir — et ce n'est pas un échec

C'est un échafaudage, pas une cage.

Le jour où votre état ne rentre plus dans `Partie`, vous écrivez le contrat nu. Les vingt-quatre entrées sont documentées au chapitre « **Quand l'échafaudage ne suffit plus** », à la fin de ce cours — et il se lira beaucoup mieux à ce moment-là, parce que vous aurez déjà vu les dix-huit fonctions à l'œuvre.

Vous n'avez rien à défaire pour passer de l'un à l'autre : les deux produisent le même module, avec les mêmes deux symboles. `RegleFacile.cpp` et `RegleContratNu.cpp` jouent **exactement le même jeu** — comparez-les, c'est l'exercice le plus instructif du cours.

Exercices

1. **Portée 2.** Autorisez la duplication vers une case à deux pas. Une seule ligne change dans `CoupsPossibles` — laquelle ?
2. **Le déplacement.** Ajoutez l'action `DEPLACER` (la case de départ se vide). Que devient le décompte des totems ?
3. **Hexagone.** Passez `g.topologie` à `NkcTopology::HexPointy`. Combien de voisins avez-vous maintenant, et quelle ligne de votre code l'a supposé ?
4. **La cascade.** Émettez `ev.Cascade(...)` quand un coup retourne au moins deux ennemis, et regardez ce que l'atelier en fait à l'écran.

Écrire une IA

Le fichier de ce chapitre : `Applications/ConquerorLab/exemples/ai/IAMinimale.cpp`. Il tient en une idée : **demandeur les coups légaux au moteur, en prendre un au hasard**. C'est nul, et c'est le but.

3.1 Pourquoi commencer par une IA nulle

Une IA aléatoire est le **plancher de mesure**. Sans elle, vous n'avez aucun point de comparaison : une IA qui gagne 70 % ne veut rien dire tant qu'on ne sait pas contre quoi.

Le premier run du banc d'essai livre déjà un enseignement de cette nature :

Le glouton bat l'aléatoire 29–6. La position se juge, l'évaluation matérielle mord. CONQUERORLAB/README.MD

C'est une information réelle sur le jeu, obtenue avec deux IA triviales. Une IA qui ne bat pas l'aléatoire ne vaut rien ; une qui la bat 55 % du temps ne vaut pas beaucoup plus.

3.2 Le point de conception central

`include/Conqueror/ConquerorAIABI.h:14-22`

```
1 POINT DE CONCEPTION CENTRAL
2 L'IA ne connaît PAS l'implementation du moteur de regles : elle recoit une
3 NkcRulesVTable et un handle opaque. Consequences voulues :
4
5 1. Le sujet A2 démarre en semaine 1 sur un module bouchon, sans attendre A1.
6 2. Quand A1 change son code interne, l'IA continue de fonctionner.
7 3. Les deux ne peuvent pas diverger sur les regles : il n'en existe qu'une
8 implementation, et l'IA la traverse.
```

Concrètement, votre IA ne sait pas ce qu'est une duplication. Elle ne connaît ni le plateau, ni la topologie, ni les paramètres. Elle demande :

`exemples/ai/IAMinimale.cpp — V_ChooseMove`

```
1 // TOUT passe par la table : on ne sait pas ce qu'on joue, on demande.
2 NkcMove moves[kMoveCap];
3 const uint32 total = R->GenerateLegalMoves(ri, st, moves, kMoveCap);
4 const uint32 n = total < kMoveCap ? total : kMoveCap;
5 if (n == 0) return 0;
```

Ce qu'il faut retenir

Vous pouvez écrire une IA **avant** que les règles existent. Vous pouvez la tester contre trois moteurs différents sans la recompiler. Et le jour où A1 change sa représentation interne, vous ne le saurez même pas.

3.3 Les neuf fonctions

Fonction	Obligatoire	Rôle
Create / Destroy	oui	cycle de vie
Configure	oui	palier, budget, graine — avant le 1 ^{er} ChooseMove
ChooseMove	oui	le cœur
GetParamsSchemaJson SetParam	/ non	réglages; renvoyer "" est légal
OnMovePlayed	non	un coup a réellement été joué
Reset	non	nouvelle partie : vider la mémoire
GetDebugJson	non	ce que l'atelier pourra afficher

3.4 ChooseMove, ligne à ligne

exemples/ai/IAMinimale.cpp — V_ChooseMove, entree

```
1 int32 V_ChooseMove(NkcAI self, const NkcRulesVTable *R, NkcRules ri,
2                   const NkcState st, NkcAIResult *out) {
3     AI *a = static_cast<AI *>(self);
4     if (!a || !R || !ri || !st || !out) return 0;
5
6     std::memset(out, 0, sizeof(*out));
7     out->move.targetLevel = -1;
8     out->move.powerId     = -1;
```

Le même piège que pour les règles

NkcMove se compare **octet à octet**. Un champ laissé au hasard rend votre coup illégal aux yeux du moteur.

Ce n'est pas strictement nécessaire quand on recopie un coup issu de GenerateLegalMoves — il est déjà propre — mais une IA qui *construit* un coup au lieu de le choisir doit absolument le faire.

exemples/ai/IAMinimale.cpp — V_ChooseMove, choix

```
1     const uint32 pick = static_cast<uint32>(NextRand(a->rng) % n);
2     out->move          = moves[pick];
3     out->simulations    = 1;
4     out->depthReached   = 0;
5     out->scoreMilli     = 0;
6     return 1;
7 }
```

Le retour est binaire : **1 = j'ai un coup**, **0 = je n'en ai aucun**. Dans le second cas l'atelier joue PASSER lui-même.

3.4.1 Le compte-rendu

NkcAIResult n'est pas décoratif : le panneau *Joueurs* l'affiche après chaque réflexion.

include/Conqueror/ConquerorAIABI.h:78-86

```
1 struct NkcAIResult {
2     NkcMove move;
3     int32 scoreMilli = 0; // -1000 = perdue, +1000 = gagnee
4     uint32 simulations = 0; // simulations/noeuds reellement faits
5     uint32 elapsedMs = 0;
6     uint32 depthReached = 0;
7     uint8 hitBudget = 0; // 1 si coupee par le budget
8     uint8 _pad[3] = {};
9 };
```

scoreMilli est en MILLIÈMES

Aucun flottant à la frontière, même pour un score. Même raison qu'ailleurs : deux plateformes doivent produire le même nombre.

3.5 Le PRNG : porté par l'instance

exemples/ai/IAMinimale.cpp — NextRand

```
1 // PRNG xorshift64* PORTE PAR L'INSTANCE, jamais global. Sans cela, deux
2 // instances d'IA jouant en parallele dans une campagne piochent dans le
3 // meme flux, et deux parties de meme graine cessent d'etre reproductibles.
4 uint64 NextRand(uint64 &s) {
5     s ^= s >> 12;
6     s ^= s << 25;
7     s ^= s >> 27;
8     return s * 2685821657736338717ull;
9 }
```

La graine arrive par Configure, et l'atelier la dérive de (partie, tour, joueur) :

src/ConquerorLab/NkcSession.h — StartThinking

```
1 // Graine derivee de (partie, tour, joueur) : deux parties de meme graine
2 // rejouent a l'identique, et deux tours ne partagent pas le meme flux.
3 c.seed = mSeed ^ (static_cast<uint64>(mView.turn) * 0x9E3779B97F4A7C15ull) ^
4     (static_cast<uint64>(mView.current) + 1ull);
```

Un static dans votre IA, et la campagne ment

Une campagne fait tourner **plusieurs instances de votre IA en parallèle**, une par worker. Un compteur static, un cache global, un générateur partagé : et vos parties cessent d'être indépendantes.

Le résultat reste *plausible* — c'est le pire des cas, parce que vous ne le verrez pas. Déclarez-le franchement dans votre fabrique : `info.isThreadSafe = 0` si vous n'en êtes pas sûr.

3.6 Le thread, et le budget

include/Conqueror/ConquerorAIABI.h:24-32

```
1 CONTRAINTE DE THREAD
2 ChooseMove est appelee depuis un THREAD WORKER, jamais depuis le thread de
3 rendu. Elle doit :
4 - ne toucher que l'etat qu'on lui passe (c'est un CLONE prive) ;
5 - ne poser aucun verrou sur une ressource partagee ;
6 - respecter `budgetMs` : au-dela, elle rend le meilleur coup trouve
7   jusqu-la. Depasser le budget fige l'interface -- c'est un echec.
```

Côté atelier, l'isolement est total :

L'IA réfléchit sur son propre moteur. Plutôt que de partager l'instance de règles avec le thread de rendu — où l'utilisateur peut à tout moment bouger un paramètre — le thread possède SA PROPRE instance, synchronisée avant chaque réflexion, et son propre état, transféré par `SerializeState/DeserializeState`. Zéro verrou, zéro course. `NkcSession.H`

L'état que vous recevez est à **vous**. Vous pouvez le cloner autant que vous voulez. Mais vous ne devez **pas** le muter directement — vous passez par `rules->ApplyMove` sur vos clones.

3.6.1 Respecter le budget

`IAMinimale` répond instantanément, la question ne se pose pas. Dès que vous faites une recherche, elle se pose à chaque nœud.

Idiome — écrit pour ce cours, d'après `ConquerorAIABI.h:113`

```
1 // Au debut de ChooseMove : noter l'instant de depart (l'IA choisit sa
2 // source de temps ; Le contrat n'en impose aucune).
3 // Dans la boucle de recherche, tous les N noeuds :
4 if (a->cfg.budgetMs != 0 && ElapsedMs(start) >= a->cfg.budgetMs) {
5     out->hitBudget = 1;
6     break; // on rend le MEILLEUR coup trouve jusqu-la
7 }
```

Deux règles :

- `budgetMs == 0` signifie **illimité**. Ne le traitez pas comme « zéro milliseconde » ;
- **ne testez pas le temps à chaque nœud** : un appel horloge par nœud coûte plus cher que le nœud. Tous les 1024 nœuds suffit.

Ce que « figer l'interface » veut dire concrètement

Le panneau *Joueurs* affiche `hitBudget` sous la forme « coupée par le budget ». Mais surtout : quand vous cliquez *Nouvelle partie* pendant une réflexion, l'atelier **attend** que votre `ChooseMove` rende la main. Une IA qui ignore son budget rend le bouton mou pendant tout ce temps.

3.7 Les paliers de difficulté

include/Conqueror/ConquerorAIABI.h:41-51

```
1  /// Cinq paliers. Reperes, pas implementation imposee : c'est au module de
2  /// decider comment il module sa force (budget de simulations, profondeur,
3  /// bruit, erreurs volontaires).
4  enum class NkcDifficulty : uint8 {
5      Easy = 0, Normal = 1, Hard = 2, Expert = 3, Apex = 4, Count = 5
6  };
```

L'IA de référence les utilise comme trois stratégies distinctes : aléatoire pur en *Facile*, glouton en *Normal*, minimax alpha-bêta de profondeur 2, 3 puis 4 pour les trois derniers.

Si votre IA ne distingue qu'un seul niveau, dites-le : `info.difficultyCount = 1`.

3.8 Évaluer une position sans connaître les règles

C'est le problème intéressant du sujet A2. Voici comment l'IA de référence s'y prend — et c'est exactement ce qu'il faut remettre en cause.

modules/ai/ConquerorAIRef.cpp — Evaluate

```
1  int32 mine = 0, theirs = 0;
2  for (uint8 p = 0; p < v.playerCount; ++p) {
3      if (p == me) mine += v.totemCount[p];
4      else        theirs += v.totemCount[p];
5  }
6
7  // Mobilité du joueur au trait, bornée pour rester bon marche.
8  NkcMove      buf[64];
9  const uint32 n      = R->GenerateLegalMoves(ri, st, buf, 64);
10 int32 mobility = static_cast<int32>(n > 64 ? 64 : n);
11 if (v.current != me) mobility = -mobility;
12
13 return ai->wMaterial * (mine - theirs) + ai->wMobility * mobility;
```

Deux termes seulement — le **matériel** et la **mobilité** — et deux poids réglables. Tout ce qu'une IA peut savoir vient de `NkcStateView` et de `GenerateLegalMoves`.

Le vrai sujet de A2

Cette évaluation est délibérément grossière. « Le glouton bat l'aléatoire 29–6 » dit que le matériel mord ; il ne dit pas que c'est le bon critère.

Une position avec beaucoup de totems mais peu de mobilité est-elle bonne ? Un totem isolé au centre vaut-il un totem en bord de plateau ? **Ce sont des questions que seule la mesure tranche** — chapitre 6.

3.9 OnMovePlayed : ce qui distingue un MCTS d'une recherche jetable

Une recherche naïve jette tout son travail à chaque tour. Un MCTS conserve son arbre : quand un coup est joué, il *descend* dans le sous-arbre correspondant et repart de là. `OnMovePlayed` est le signal qui le lui permet, `Reset` celui qui lui dit de tout jeter.

Attention en campagne

Une campagne réutilise la même instance d'IA d'une partie à l'autre. Sans Reset correct, l'arbre de la partie précédente contamine la suivante — et vos mesures deviennent dépendantes de l'ordre des parties.

Le lanceur de campagne appelle Configure puis Reset au début de chaque partie ; à vous d'honorer le second.

3.10 Votre première IA, en quatre gestes

1. copiez `exemples/ai/IAMinimale.cpp` vers `Build/ConquerorLab/ai/mon\_ia.cpp`;
2. changez le nom dans `FillFactory`;
3. sauvegardez, attendez une seconde, ouvrez le panneau *Joueurs* : votre IA est dans la liste des pilotes ;
4. mettez-la sur le siège 1, l'IA de référence sur le siège 0, et lancez une campagne de 200 parties.

Vous venez de mesurer votre plancher. Tout ce que vous ferez ensuite se compare à ce chiffre.

1 — Le glouton

Transformez `IAMinimale` en glouton : pour chaque coup légal, clonez l'état, appliquez le coup, comptez vos totems, gardez le meilleur. Vous aurez besoin de `R->CreateState`, `CloneState`, `ApplyMove`, `GetView`, `DestroyState`. Mesurez-le contre l'aléatoire sur 200 parties.

2 — Le poids réglable

Exposez un paramètre `poids_mobilite` via `GetParamsSchemaJson` (même format que pour les règles). Vérifiez qu'il apparaît dans l'interface, puis faites-le varier et mesurez : y a-t-il une valeur optimale, ou le résultat est-il plat ?

3 — Le budget

Ajoutez une recherche à profondeur 3, puis un test de budget tous les 1024 nœuds. Réglez le budget à 5 ms dans le panneau *Joueurs* et vérifiez que `hitBudget` passe bien à 1. Que se passe-t-il si vous oubliez le test ?

4 — Le piège du static

Ajoutez un `static int compteur = 0` ; dans votre `ChooseMove` et faites-le intervenir dans le choix du coup. Lancez une campagne sur 8 threads, deux fois, avec la même graine. Les résultats sont-ils identiques ? Expliquez.

Fabriquer une grille en JSON

Ce chapitre est le plus court, et c'est significatif : **ajouter un plateau ne demande pas une ligne de code**. Le chapitre suivant montre la voie inverse — un plateau entièrement décrit en C++ — et explique quand la préférer.

4.1 Pourquoi le plateau est une donnée

REGLES_COMPLETES_v2.md:93-107

```
1 Le plateau n'est pas une constante du code : c'est un descripteur serialisable,
2 editable a la souris dans le moteur de test.
3
4 BoardDesc {
5     topology      : HEX_POINTY | HEX_FLAT | SQUARE_4 | SQUARE_8
6     cells         : liste de coordonnees valides           // definit la forme reelle
7     blocked       : liste de coordonnees bloquee
8     starts        : [ { player_index, coord, level } ] // positions initiales
9     min_players   : entier
10    max_players   : entier
11 }
```

Les formes prévues pour Conqueror — rectangle, hexagone, plus, diamant — ne sont pas dans le moteur : ce sont des fichiers. La raison est pratique. Le palier 0 doit répondre à « quelle taille de plateau ? quel voisinage ? quel avantage au premier joueur ? » : trois questions qui demandent d'essayer vingt plateaux. Si chaque essai coûte une recompilation, on en essaiera trois.

4.2 Où déposer le fichier

`Build/ConquerorLab/boards/ma_grille.json`. Le chemin exact est affiché dans le panneau *Règles*, section **Plateau**. Le fichier apparaît dans la liste déroulante à la seconde suivante — **sans compilation**, puisqu'il n'y a rien à compiler.

Au premier lancement, l'atelier y écrit un exemple :

Écrit un exemple si le dossier est vide, à partir du plateau que le moteur chargé expose. **Le format documenté vaut mieux qu'un format décrit** : celui-ci est forcément valide, puisqu'il vient du module.

NKCBOARDLIBRARY.H

4.3 Le format, sur un exemple minuscule

Un plateau carré 3×3 , deux joueurs en diagonale, une case bloquée au centre :

Écrit pour ce cours — format de LoadBoardJson

```
1 {
2     "topology": "SQUARE_4",
3     "cells": [[0,0],[1,0],[2,0],
```

```

4         [0,1],[1,1],[2,1],
5         [0,2],[1,2],[2,2]],
6     "blocked": [[1,1]],
7     "starts": [
8         {"player": 0, "q": 0, "r": 0, "level": 0},
9         {"player": 1, "q": 2, "r": 2, "level": 0}
10    ],
11     "min_players": 2,
12     "max_players": 2
13 }

```

Champ	Rôle	Piège
topology	hex ou carré	fixe le voisinage <i>et</i> l'interprétation de (q, r)
cells	la forme réelle	une coordonnée absente d'ici n'existe pas : c'est le hors-plateau
blocked	cases inoccupables	doivent aussi figurer dans cells
starts	totems de départ	player au-delà de max_players est ignoré
min_players	bornes du plateau	informatif, avec max_players : c'est le moteur qui décide quoi en faire

cells définit la forme, pas un rectangle englobant

C'est ce qui permet des plateaux en croix, en diamant, ou troués. Ne listez **que** les cases qui existent. Le moteur de référence teste l'appartenance par recherche dans cette liste : une coordonnée absente est du hors-plateau, exactement comme si elle était au-delà du bord.

4.4 Hexagone : la conversion qui piège

En hexagone, les coordonnées sont **axiales** (q, r) , pas des lignes et des colonnes. Le moteur de référence construit son 6×7 ainsi :

modules/rules/ConquerorRulesV2.cpp — BuildDefaultBoard

```

1  for (int32 row = 0; row < 7; ++row) {
2      for (int32 col = 0; col < 6; ++col) {
3          NkcCoord c;
4          c.q = static_cast<int16>(col - (row >> 1)); // odd-r -> axial
5          c.r = static_cast<int16>(row);
6          B.cells[B.cellCount++] = c;
7      }
8  }

```

La ligne qui compte est `col - (row >> 1)` : c'est la conversion **odd-r vers axial**. Si vous écrivez un générateur de plateau hexagonal, c'est là que vous vous tromperez. Le symptôme est net et reconnaissable : le plateau s'affiche en **losange penché** au lieu d'un rectangle d'hexagones.

Le raccourci qui évite l'erreur

Ne calculez pas vos coordonnées à la main. Chargez le plateau par défaut, cliquez « **Exporter le plateau courant** » dans le panneau *Règles*, et modifiez le fichier obtenu. Vous partez d'une géométrie juste.

4.5 La symétrie n'est pas une coquetterie

Le placement initial doit être symétrique sous une isométrie du plateau. Le moteur de test doit refuser de lancer un batch d'équilibrage sur un plateau asymétrique, ou signaler l'asymétrie dans le rapport. Sans cela, **tout écart de winrate mesuré est ininterprétable** : on ne saura pas s'il vient des règles ou du plateau. REGLES_COMPLETES_V2.MD :133

Cette vérification n'existe pas encore dans l'atelier

Le document l'exige; l'atelier ne la fait pas. **C'est à vous de vérifier la symétrie de vos plateaux à l'œil** avant de lancer une campagne dessus.

Il y a toutefois un garde-fou partiel : la campagne **inverse les côtés une partie sur deux** (chapitre 6), ce qui sépare l'avantage de position de l'avantage de stratégie. Cela compense un plateau asymétrique *entre les deux camps*, mais pas un plateau qui avantagerait structurellement le premier joueur.

4.6 Quand le moteur refuse votre fichier

L'atelier ne parse pas le JSON : il passe la chaîne telle quelle à LoadBoardJson. C'est donc le **module** qui accepte ou refuse.

src/ConquerorLab/NkcBoardLibrary.h — LoadInto

```
1 if (!vt.LoadBoardJson(inst, text.CStr())) {
2     mMsg = "Le moteur a REFUSE ce plateau : ";
3     mMsg += mFiles[idx].name;
4     mMsg += " (cellules manquantes ou JSON invalide)";
5     return false;
6 }
```

Ce message s'affiche en rouge dans le panneau *Règles*. Trois causes, par ordre de fréquence :

1. **cells vide ou absent** — le moteur de référence refuse explicitement un plateau sans cases;
2. **JSON malformé** — le scanner des modules est minimal : ni commentaires, ni virgules traînantes;
3. **le module ne sait pas charger de plateau** — *RegleContratNu.cpp* renvoie toujours 0 : son plateau est figé dans le code. C'est un renoncement assumé pour un exemple.

Conséquence heureuse de « l'atelier ne parse rien »

Si votre moteur accepte un champ supplémentaire — disons "artefacts":[{"q":1,"r":2,"id":7}] — il le verra **sans qu'on touche à l'atelier**. Le format du plateau vous appartient; le contrat ne fixe qu'un socle commun.

4.7 Ce que l'atelier ne fait pas

Le document de règles parle d'un plateau «éditable à la souris». **Cet éditeur n'existe pas.** Aujourd'hui, on écrit le JSON à la main, ou on exporte puis on modifie.

C'est une limite réelle et assumée : l'éditeur graphique viendra quand quelqu'un aura assez souffert du fichier texte pour savoir de quoi il a besoin.

1 — Le plateau troué

Exportez le plateau par défaut, puis retirez huit cases au centre pour en faire un anneau. Chargez-le. Que devient la durée moyenne d'une partie sur 200 parties IA contre IA ? Expliquez.

2 — Carré contre hexagone

Fabriquez un plateau SQUARE_8 de 42 cases (6×7) et comparez-le à l'hexagone 6×7 par défaut, à **même nombre de cases**. Lancez 500 parties sur chacun. Le taux de nuls change-t-il ? Le nombre de coups ? Que dit ce résultat sur l'effet du voisinage ?

3 — L'asymétrie visible

Fabriquez volontairement un plateau asymétrique : trois totems de départ au joueur 0, deux au joueur 1. Lancez 400 parties *avec* l'inversion des côtés, puis 400 *sans*. Comparez et expliquez précisément ce que l'inversion a corrigé — et ce qu'elle n'a pas corrigé.

4.8 La forme des cellules

Une grille carrée doit-elle se dessiner en carrés ? Non — et les séparer est instructif.

boards/_generer.py — plateau()

```
1 def plateau(topology, cells, starts, blocked=None, cell_shape=None):
2     """`cell_shape` est de la PRESENTATION : "SQUARE", "HEX" ou "CIRCLE".
3
4     Elle ne touche ni au voisinage, ni aux coups legaux, ni au resultat -
5     c'est justement pourquoi elle n'est PAS dans le contrat : le moteur ne
6     la lit jamais, seul l'atelier la regarde pour dessiner.
7     """
```

Trois formes livrées : **carrée**, **hexagonale**, **ronde**. Le champ est facultatif ; absent, l'atelier déduit la forme de la topologie, comme avant. Le panneau *Règles* offre en plus un réglage **Forme des cellules** qui prime sur le fichier — pour comparer sans rien réécrire.

Trois sources, un ordre de priorité

1. le module, s'il fournit `GetCellShape` (chapitre 5) — il a le dernier mot sur *sa* géométrie ;
2. votre choix dans le panneau *Règles* ;
3. ce que déclare le fichier `.json`.

4.9 La bibliothèque livrée

Quatorze plateaux, présents dès le premier lancement.

Forme	Fichiers
hexagonal	hexagone_6x7 (référence), 4x5, 9x8, coeur_bloque, plat
rectangulaire	rectangle_8x6, carre_8x8_diagonales
en croix	plus
parallélogramme	parallogramme_6x7, parallogramme_8x5
libre	diamant
formes de cellule	rectangle_8x6_rond, hexagone_6x7_rond, carre_8x8_hexagones

Le parallélogramme mérite un mot : c'est `hex_rect` **sans** la correction - (`row >> 1`), autrement dit la forme qu'on obtient quand on oublie la conversion `odd-r` → `axial`. La livrer comme un plateau à part entière vaut mieux que de la laisser apparaître comme un bug — la reconnaître à l'écran fait comprendre du même coup ce que corrige `hex_rect`.

Tous sont produits par [boards/_generer.py](#), jamais écrits à la main : la symétrie des départs exigée par REGLES §4.2 y est **calculée par réflexion**. Un plateau asymétrique rend tout écart de `winrate` ininterprétable.

Définir sa grille en C++

Le chapitre précédent décrivait un plateau dans un fichier JSON. Ce chapitre montre l'autre voie — et corrige au passage un malentendu répandu.

5.1 Le malentendu : «je suis obligé de passer par du JSON»

Non. **Vous ne l'avez jamais été.** Le contrat ne vous impose ni format de plateau, ni fichier, ni même l'existence d'un fichier. Regardez ce que `LoadBoardJson` vaut dans `exemples/rules/RegleContratNu.cpp` :

`exemples/rules/RegleContratNu.cpp — V_LoadBoardJson`

```
1  /// Plateau fige : on REFUSE tout chargement. L'atelier l'affiche proprement
2  /// (« Le moteur a REFUSE ce plateau ») au lieu de faire semblant.
3  int32 V_LoadBoardJson(NkcRules, const char *) { return 0; }
```

Ce module refuse tout JSON, et il fonctionne parfaitement dans l'atelier. Son plateau est construit en C++, dans `Create` :

`exemples/rules/RegleContratNu.cpp — V_Create`

```
1  for (int32 y = 0; y < kSide; ++y)
2      for (int32 x = 0; x < kSide; ++x) {
3          R->coords[y * kSide + x].q = static_cast<int16>(x);
4          R->coords[y * kSide + x].r = static_cast<int16>(y);
5      }
```

Trois choses ont TOUJOURS été à vous

La forme du plateau quelles coordonnées existent : c'est le tableau `coords` que vous exposez dans `GetView`. Une coordonnée absente est du hors-plateau, point.

Le voisinage qui touche qui : c'est *votre* code dans `GenerateLegalMoves`. `ConquerorGeometry.h` n'est qu'une **commodité**, pas une obligation — vous pouvez ne pas l'inclure.

Les cases bloquées c'est `kCellBlocked` dans le champ `owner` de la vue. Rien d'autre.

Le JSON n'existe que pour qu'un *designer* — quelqu'un qui ne compile pas — puisse essayer vingt formes dans l'après-midi.

5.2 Ce qui vous échappait vraiment : la projection écran

Une seule chose ne vous appartenait pas : **où l'atelier dessine vos cellules, et quelle forme il leur donne.** Il le déduisait de `topology`, donc de quatre valeurs possibles — deux hexagones, deux carrés.

Conséquence : un plateau que vos règles savaient parfaitement jouer pouvait être **impossible à afficher**. Un plateau circulaire, en triangles, en octogones, à cellules de tailles inégales : le moteur tournait, l'écran mentait.

L'ABI 3 rend la main au module.

include/Conqueror/ConquerorRulesABI.h — bloc geometrie (ABI 3)

```
1 // TOUT CE BLOC EST OPTIONNEL : laisser a nullptr fait retomber l'atelier
2 // sur la projection standard de `topology`.
3
4 /// Centre de la cellule `c`, en unites de cellule (1.0 = un « pas » de
5 /// grille). L'atelier cadre et met a l'echelle ensuite.
6 int32 (*GetCellCenter)(NkcRules self, NkcCoord c, float32 *outXY) = nullptr;
7
8 /// Contour de la cellule `c` : `capPoints` paires (x, y) au plus, en unites
9 /// de cellule, RELATIVES au centre. Renvoie le nombre de points écrits.
10 uint32 (*GetCellShape)(NkcRules self, NkcCoord c, float32 *outXY,
11                        uint32 capPoints) = nullptr;
```

Ces flottants ne violent pas «zéro flottant»

L'exigence §17.1 porte sur la **logique de règles** : ce qui décide d'un coup, alimente un état, entre dans une empreinte. Ces deux fonctions ne font que de la **présentation** — l'atelier les appelle pour dessiner, jamais pour jouer.

Le repère est simple : si votre HashState ne les voit pas, elles sont du bon côté de la frontière.

5.3 Un plateau circulaire, entièrement en C++

Le fichier : Applications/ConquerorLab/exemples/rules/GrilleLibre.cpp. Trois anneaux concentriques, 19 cellules en secteurs. Ni hexagone, ni carré.

exemples/rules/GrilleLibre.cpp — la forme, en entiers

```
1 constexpr int32 kRings          = 3;
2 constexpr int32 kRingSize[kRings] = {1, 6, 12};
3 constexpr int32 kCells          = 1 + 6 + 12; // 19
4 constexpr int32 kRingStart[kRings] = {0, 1, 7}; // index du 1er de chaque anneau
5
6 int32 IndexOf(NkcCoord c) {
7     if (c.q < 0 || c.q >= kRings) return -1;
8     if (c.r < 0 || c.r >= kRingSize[c.q]) return -1;
9     return kRingStart[c.q] + c.r;
10 }
```

Une coordonnée vaut ici (q = anneau, r = index dans l'anneau). C'est *notre* convention : le contrat ne dit rien du sens de q et r hors topologie standard, et c'est exactement ce qui rend la chose possible.

5.3.1 Le voisinage, défini par nous

exemples/rules/GrilleLibre.cpp — Neighbors

```
1 uint32 Neighbors(NkcCoord c, NkcCoord *out, uint32 cap) {
2     uint32 n = 0;
3     auto push = [&](int32 ring, int32 idx) {
4         if (ring < 0 || ring >= kRings) return;
5         const int32 sz = kRingSize[ring];
6         const int32 r = ((idx % sz) + sz) % sz; // modulo circulaire
7         if (n < cap) { out[n].q = (int16)ring; out[n].r = (int16)r; }
8         ++n;
9     };
10
11     if (c.q == 0) { // Le centre touche tout l'anneau 1
12         for (int32 i = 0; i < kRingSize[1]; ++i) push(1, i);
13         return n;
14     }
15     push(c.q, c.r - 1); // voisin arriere sur l'anneau
16     push(c.q, c.r + 1); // voisin avant sur l'anneau
17     if (c.q == 1) {
18         push(0, 0); // vers le centre
19         push(2, c.r * 2); // vers l'exterieur : DEUX cellules
20         push(2, c.r * 2 + 1);
21     } else { // anneau 2
22         push(1, c.r / 2); // vers l'interieur : UNE cellule
23     }
24     return n;
25 }
```

C'est le cœur de la démonstration : `ConquerorGeometry.h` n'est même pas inclus dans ce fichier. Aucune topologie du contrat ne sait exprimer «le centre touche tout l'anneau 1», ni «une cellule de l'anneau 1 touche deux cellules de l'anneau 2». Ici, si — et le tout reste **entièrement entier**, donc le jeu bit-à-bit tient.

Ce qu'il faut retenir

L'ordre reste **normatif** : `push` est appelé dans un ordre fixe, et c'est lui qui fixe l'ordre de génération des coups. Ne le réordonnez pas.

5.3.2 Où dessiner : `GetCellCenter`

exemples/rules/GrilleLibre.cpp — `V_GetCellCenter`

```
1 int32 V_GetCellCenter(NkcRules, NkcCoord c, float32 *outXY) {
2     if (!outXY) return 0;
3     if (c.q < 0 || c.q >= kRings) return 0;
4     if (c.q == 0) { outXY[0] = 0.f; outXY[1] = 0.f; return 1; }
5     const float32 a = CellAngle(c);
6     const float32 R = static_cast<float32>(c.q); // anneau 1 -> rayon 1
7     outXY[0] = R * std::cos(a);
8     outXY[1] = R * std::sin(a);
9     return 1;
10 }
```

Des coordonnées polaires, en **unités de cellule**. Vous ne vous souciez ni du zoom, ni des pixels, ni de la taille de la fenêtre : l'atelier calcule la boîte englobante réelle de vos centres et met tout à l'échelle.

Renvoyer 0 signifie «je ne sais pas placer celle-ci» — l’atelier retombe alors sur la topologie pour cette cellule seulement. Vous pouvez donc ne personnaliser qu’une partie du plateau.

5.3.3 Quelle forme : GetCellShape

exemples/rules/GrilleLibre.cpp — V_GetCellShape, secteur d’anneau

```
1 // 4 points sur l'arc exterieur, puis 4 sur l'interieur en sens inverse.
2 for (uint32 i = 0; i < seg; ++i) {
3     const float32 t = static_cast<float32>(i) / static_cast<float32>(seg - 1);
4     const float32 a = a0 - half + 2.f * half * t;
5     outXY[n * 2] = (R + 0.46f) * std::cos(a) - cx;
6     outXY[n * 2 + 1] = (R + 0.46f) * std::sin(a) - cy;
7     ++n;
8 }
9 for (uint32 i = 0; i < seg; ++i) {
10    const float32 t = static_cast<float32>(seg - 1 - i) / static_cast<float32>(seg - 1);
11    const float32 a = a0 - half + 2.f * half * t;
12    outXY[n * 2] = (R - 0.46f) * std::cos(a) - cx;
13    outXY[n * 2 + 1] = (R - 0.46f) * std::sin(a) - cy;
14    ++n;
15 }
```

Un polygone relatif au centre, en unités de cellule. Huit points suffisent à faire un secteur d’anneau convaincant; kMaxCellPoints vaut 12.

Le polygone doit rester convexe

L’atelier remplit vos cellules par un éventail de triangles depuis le premier sommet — c’est la primitive dont dispose NkGuiDrawList, qui n’a pas de remplissage de polygone quelconque. Un contour concave se remplira de travers.
Un secteur d’anneau assez fin passe très bien; une cellule en U, non.

5.3.4 Le déclarer aux autres : GetNeighbors

Votre voisinage est écrit, il marche, vos coups sont bons. Il reste un problème : **personne d’autre ne le connaît**.

- une IA qui évalue une position («combien d’ennemis touche cette case?») n’avait que view.topology pour le deviner. Sur une grille libre, elle devinait **faux, en silence**;
- l’atelier ne pouvait pas montrer le voisinage à l’écran, donc un stagiaire qui se trompait d’adjacence n’avait aucun moyen de le **voir**.

D’où une troisième entrée — et celle-ci est **entièrement entière**, c’est de la règle, pas du dessin :

exemples/rules/GrilleLibre.cpp — V_GetNeighbors

```
1 /// Le voisinage, DECLARE. Sans cette entree, une IA qui evalue une position
2 /// sur ce plateau devinerait l'adjacence a partir de `topology` - et
3 /// devinerait faux, en silence.
4 uint32 V_GetNeighbors(NkcRules, NkcCoord c, NkcCoord *out, uint32 cap) {
5     return Neighbors(c, out, cap);
6 }
```

Une ligne, puisque la fonction existait déjà. Vérifié :

```
1 GetNeighbors(0,0) -> 6 voisins : (1,0) (1,1) (1,2) (1,3) (1,4) (1,5)
```

Le centre touche tout l’anneau 1 — aucune topologie du contrat ne sait exprimer cela.

Le bouton « Voisinage » du panneau Plateau

Il trace, au survol, les liens de la case vers ses voisins. C’est le **seul moyen de vérifier une adjacence à l’œil**, et il lit `GetNeighbors`. Sans lui, une erreur d’adjacence se découvre par un coup légal inexplicable, trois heures plus tard.

5.4 Comment l’atelier s’y adapte

Tout passe par un objet minuscule, reconstruit à chaque image :

src/ConquerorLab/NkcBoardRender.h — le projecteur

```
1 struct NkcProjector {
2     const NkcRulesVTable *vt      = nullptr;
3     NkcRules                inst   = nullptr;
4     NkcTopology             topology = NkcTopology::HexPointy;
5     bool                    custom  = false; // Le module declare sa geometrie
6 };
```

Pourquoi il n’a aucun état persistant

Le rendre persistant obligerait à l’invalidier quand le module change — et c’est exactement le genre d’invalidation qu’on oublie. Le reconstruire coûte deux affectations et une comparaison de pointeur.

Une conséquence à connaître, côté picking :

src/ConquerorLab/NkcBoardRender.h — ProjPickCell

```
1 // Deux chemins, et c'est assume :
2 // - geometrie standard : inversion analytique en O(1) (arrondi cube),
3 //   exacte jusqu'aux aretes ;
4 // - geometrie du module : aucune inverse n'existe, on cherche Le centre
5 //   Le plus proche parmi les cases. O(n) sur quelques dizaines de cases
6 //   ne se mesure pas, et c'est la seule methode qui marche pour une
7 //   grille quelconque.
```

Le clic sur une grille libre désigne donc la cellule dont le *centre* est le plus proche, à condition d’être à portée. Sur des cellules très allongées, le picking peut sembler légèrement décalé près des bords : c’est le prix d’une géométrie arbitraire, et il est assumé.

5.5 JSON ou C++ : lequel choisir

Prenez le JSON quand...	Prenez le C++ quand...
la forme se décrit par une liste de coordonnées	la forme se décrit par une <i>formule</i> (anneaux, spirale, fractale)
un designer doit pouvoir l'essayer sans compiler	le voisinage n'est pas géométrique (portails, torus, graphe quelconque)
vous voulez vingt variantes de la même idée	les cellules n'ont ni la même taille ni la même forme

Rien n'interdit les deux : votre `LoadBoardJson` peut lire la liste des cases, pendant que `GetCellCenter` en calcule la position. C'est même souvent le bon partage — la *forme* en donnée, la *projection* en code.

5.6 État vérifié

Banc d'essai, 2026-08-07 — GrilleLibre contre IAMinimale

```
1      regles : GrilleLibre (3 anneaux) 1.0.0 (palier 0)
2      IA      : IAMinimale 1.0.0
3      plateau : 19 cases, 2 joueurs
4      OK      l'IA n'a jamais produit de coup illegal
5      OK      la partie se termine d'elle-meme      -> 17 coups, vainqueur 0, 10/9
6      OK      le journal se rejoue integralement
7      OK      rejeu deterministe : empreinte identique -> 15623799979863270834
8      OK      « aucun coup legal » equivaut a « joueur bloque »
9      === 13 verifications OK, 3 echecs ===
```

Les trois échecs sont **attendus** : le banc d'essai vérifie en dur le plateau du moteur de *référence* («42 cases», «2 totems par joueur», et la présence du paramètre `portee_duplication`). Un plateau de 19 cellules en anneaux n'a aucune de ces propriétés. Tout ce qui porte sur le **contrat** passe.

Une leçon de méthode, au passage

Un test qui code en dur les valeurs d'une implémentation particulière n'est pas un test du contrat. Si vous reprenez `tests/NkcAbiHarness.cpp` pour votre module, ces trois vérifications-là sont à adapter — les treize autres, non.

1 — Le tore

Partez de `RegleContratNu.cpp` et rendez son carré 5×5 *torique* : la colonne 4 touche la colonne 0, la ligne 4 touche la ligne 0. Vous n'avez à toucher que la génération des coups. Le plateau reste affiché à plat — que manque-t-il pour *voir* le repliement ? Est-ce grave ?

2 — Le triangle

Écrivez un plateau en cellules **triangulaires** : un grand triangle subdivisé, où chaque petite cellule touche ses trois voisines de côté. Vous aurez besoin des deux orientations (pointe en haut, pointe en bas) — c'est le vrai exercice. `GetCellShape` doit renvoyer trois points.

3 — La question de fond

GrilleLibre déclare `supportsHex = 0` et `supportsSquare = 0`. Cherchez dans l'atelier qui lit ces deux champs. Que faudrait-il en faire ? Proposez une réponse en trois lignes — et vérifiez si le code actuel la respecte.

Se servir de l'atelier

Les chapitres précédents produisaient du code. Celui-ci apprend à s'en servir — et surtout à lire un résultat sans se tromper, ce qui est plus difficile que de l'obtenir.

6.1 Les sept panneaux

Tous sont dockables : on les déplace, on les empile en onglets, on les ferme.

Panneau	Ce qu'il fait
Plateau	l'état, le picking, les coups légaux, la menace, le dernier coup, le score
Règles	la grille + tous les paramètres du moteur, auto-générés
Joueurs	qui tient chaque siège, à quelle force, avec quel budget
Modules	les .cpp détectés, leur statut, la sortie du compilateur
Journal	la suite des coups, leur empreinte, le curseur de rejeu
Métriques	la campagne IA contre IA, et ce qu'elle répond
Sortie	ce que votre code écrit quand il tourne (NKC_LOG_*)

Un panneau qui n'est pas l'onglet actif ne se dessine pas

C'est le fonctionnement normal du docking. Conséquence : ouvrez le panneau *Métriques* pour voir la progression d'une campagne. La campagne, elle, tourne quand même — elle vit sur ses propres threads, pas dans l'interface.

6.2 Lire le plateau

- **disques colorés** : les totems. Le *niveau* se lit à la *taille* et au liseré, jamais à la seule couleur — celle-ci porte déjà le propriétaire;
- **anneaux verts** : les destinations légales du totem sélectionné;
- **anneaux rouges** : les totems ennemis que le coup *survolé* retournerait;
- **anneau orange qui pulse** : le dernier coup joué, une seconde environ;
- « **CASCADE ×N** » au centre : deux totems ou plus retournés d'un coup.

Les anneaux rouges sont la lecture centrale du jeu : « quelle surface de contact suis-je en train d'offrir ? ». Ils ne sont pas déduits des règles — l'atelier simule le coup et lit les événements. Ils restent donc justes même quand vous changez la règle de transformation.

6.2.1 La barre d'état, et les boutons

La barre dit à tout instant ce que l'atelier attend. **Relisez-la avant de chercher un bug.**

Bouton	Effet
Nouvelle partie	rejoue depuis le début, mêmes réglages, même graine
Lecture / Pause	la partie avance-t-elle toute seule ?
Pas à pas	un seul coup d'IA, même en pause
IA vs IA	tous les sièges à l'IA + lecture : la configuration de mesure
Siège : Humain / IA	bascule le siège au trait
Voisinage	trace les liens d'adjacence de la case survolée (mise au point)

6.3 Le panneau Sortie : vos propres traces

Le panneau **Modules** montre ce que dit le *compilateur* ; celui-ci montre ce que dit votre *code* une fois qu'il tourne. Les deux sont nécessaires, et les mélanger rendrait les deux illisibles.

- **Niveau minimal** filtre : passez à **WARN** quand vous cherchez une anomalie dans un flot de **INFO** ;
- les lignes identiques **consécutives** sont fusionnées en (xN) — une boucle qui répète la même trace mille fois reste lisible ;
- le plus récent est **en haut**, donc toujours visible sans défiler ;
- le tampon est borné à **4096 lignes**, et le panneau **affiche ce qu'il a perdu**. Un journal qui perd des lignes en silence fait chercher un bug qui n'existe pas.

Les modules internes n'y écrivent pas

ConquerorRulesV2 (interne) et AIRef (interne) sont compilés *dans* l'application : ils partagent son journal, et leurs traces vont dans la console, pas ici. Le panneau Sortie est là pour **votre** code.

6.4 Reproduire un bug : le journal

C'est la fonction la plus utile de l'atelier, et la moins spectaculaire.

Le panneau *Journal* liste les coups avec, à droite de chacun, **l'empreinte de l'état après ce coup**. Le bouton **Copier** met dans le presse-papiers une trace rejouable : graine, nombre de joueurs, un coup par ligne, empreinte finale.

Comment on s'en sert vraiment

Votre moteur produit un résultat différent sur Linux et sur Windows. Vous copiez le journal des deux côtés, vous comparez les colonnes d'empreintes, et vous trouvez **la première ligne qui diffère**. Ce coup-là, et pas un autre, contient votre non-déterminisme. Sans les empreintes intermédiaires, vous sauriez seulement que les états finaux diffèrent — c'est-à-dire presque rien.

Le bouton **Copier** produit une trace **auto-descriptive** :

Entete d'une trace copiee

```
1 # ConquerorLab – trace rejouable
2 # graine          20260807
```

```

3 # joueurs      2
4 # regles      ConquerorRulesV2 (interne)
5 # siege 0     Humain
6 # siege 1     AIRef (1.0.0)
7 # plateau     hexagone_6x7.json
8 #
9 # joueur action de_q de_r vers_q vers_r empreinte

```

La graine et les coups ne suffisaient pas : rejouée avec un autre moteur ou une autre IA, la même liste donne un autre résultat, et on cherche un bug là où il n’y en a pas. Vous êtes deux à travailler en parallèle — sans cet entête, un rapport ne dit même pas de qui vient le code qu’il incrimine.

Le curseur de rejou **reconstruit** la position (Setup + N coups) au lieu de « défaire » le dernier coup : le contrat n’impose au moteur aucune opération inverse, et une annulation approximative donnerait un état qui n’a jamais existé.

6.5 Deux garde-fous contre les erreurs silencieuses

Ce sont les plus dangereuses : rien ne plante, les chiffres sortent, et ils sont faux.

6.5.1 L’incohérence voisinage / coups légaux

Un moteur déclare son voisinage (GetNeighbors) et génère ses coups. Rien n’oblige les deux à s’accorder. Quand ils divergent, **rien ne se plaint** — mais une IA qui compte « combien d’ennemis touche cette case » se trompe, et joue simplement moins bien. Une semaine perdue à chercher dans l’évaluation un bug qui est dans la géométrie.

L’atelier vérifie donc, à chaque nouvelle partie, qu’un coup DUPLIQUER va bien vers une case que GetNeighbors déclare voisine. Sinon, bandeau rouge en tête du panneau *Modules* :

Ce que vous lisez si les deux se contredisent

```

1 INCOHERENCE : 3 coup(s) DUPLIQUER visent une case que GetNeighbors ne
2 declare pas voisine — p.ex. (2,1) -> (4,1). Ton generateur de coups et ton
3 voisinage ne disent pas la meme chose ; toute IA qui evalue l'adjacence se
4 trompera.

```

Il ne dit pas lequel des deux a tort — il signale la contradiction, ce qui suffit à la faire chercher.

6.5.2 La signature de configuration

Deux chiffres, et aucun moyen de les comparer

```

1 campagne du matin : 62 % pour le camp 0
2 campagne du soir : 51 % pour le camp 0

```

Que s’est-il passé ? A1 a changé un paramètre ? A2 son évaluation ? Les deux ? Sans réponse, ces deux nombres ne veulent **rien** dire — et on ne peut même pas savoir qu’ils ne veulent rien dire.

Le panneau *Métriques* affiche donc, **au-dessus** des chiffres, tout ce qui les a produits : moteur, IA de chaque siège, paliers, budget, plateau, valeurs de **tous** les paramètres, plus un identifiant court. Et il compare avec la campagne précédente :

Le bandeau qui évite une conclusion fautive

- 1 Depuis la campagne précédente, les PARAMETRES, les IA ont changé.
- 2 Les deux résultats ne se comparent pas.

Pourquoi ça marche là où la discipline échoue

« Une variable à la fois » est la bonne règle. Mais une discipline qu'on ne peut pas **vérifier après coup** n'en est pas une : personne ne se souvient, une semaine plus tard, de ce qui avait bougé.

On n'empêche personne de changer deux choses ; on rend impossible de ne pas s'en apercevoir. C'est la seule forme de garde-fou qui tienne entre deux personnes.

6.6 La campagne : ce qu'elle mesure, et comment

Panneau *Métriques*. Réglez le nombre de parties, les threads, le budget, les IA de chaque camp, puis **Lancer la campagne**.

6.6.1 L'inversion des côtés

src/ConquerorLab/NkcBatch.h — en-tête

```
1 2. LES COTES SONT INVERSEES une partie sur deux. Sans cela, tout écart
2 mesure mélange « cette IA est meilleure » et « le premier joueur est
3 avantage » -- et REGLES 15 pose justement l'avantage au premier joueur
4 comme LA question du palier 0. On mesure les deux séparément.
```

Concrètement : les parties $2n$ et $2n + 1$ partagent la **même graine** et n'échangent que les sièges. C'est cet appariement qui isole l'avantage de position. D'où deux tuiles distinctes :

- **Camp 0 / Camp 1** : quelle *stratégie* gagne, sièges confondus ;
- **Siège 1 gagne** : quel *siège* gagne, stratégies confondues. C'est l'avantage au premier joueur.

L'inversion ne s'applique qu'à deux joueurs

À 2 joueurs l'inversion échange 0 et 1 ; au-delà on ne l'applique pas — une permutation cyclique à 3–4 joueurs ne s'apparie plus deux à deux. NKC.BATCH.H, SEATTOCONFIG

À 3 ou 4 joueurs, les winrates par camp **mélangent** effet de stratégie et effet de siège. Tenez-en compte avant de conclure quoi que ce soit.

6.6.2 Les tuiles rouges

Coupées (max_tours) doit valoir **zéro**. Dès que la tuile est rouge, des parties ne se terminent pas d'elles-mêmes — et « un taux non nul signale une pathologie des règles, pas un réglage à monter ».

Coups illégaux doit valoir zéro aussi. Un chiffre non nul veut dire qu'une IA a proposé un coup que le moteur refuse — presque toujours un memset manquant, parfois une IA qui construit un coup au lieu de le choisir.

6.6.3 Les deux histogrammes

Usage des actions — cinq barres : *aucune*, *dupliquer*, *fusionner*, *pouvoir*, *passer*. La barre *aucune* doit rester à zéro : une valeur non nulle serait un bug du générateur de coups, pas un détail à masquer.

Une action jamais jouée est une action à supprimer ou à rendre attractive. C'est ce raisonnement qui a tué l'action SAUTER :

L'action SAUTER n'est jamais jouée : +0 totem, une seule cible, conquête impossible — dominée par DUPLIQUER dans toute position. **Supprimée** du jeu de base. Revient comme *pouvoir*, où elle a un coût.

REGLES_COMPLETES_V2.MD :57

Durée des parties — une barre par tranche de dix coups. Une distribution très étalée, ou un pic contre la dernière barre, signale des parties qui traînent.

6.7 Une méthode de mesure qui tient

Quatre règles, apprises à leurs dépens par d'autres.

1. **Une variable à la fois.** Changez *portee_duplication*, mesurez, notez. Puis remettez-la et changez autre chose. Deux variables ensemble donnent un résultat que rien ne permet d'attribuer.
2. **Assez de parties.** Sur 20 parties, un écart de 10 points n'est que du bruit. Comptez 200 pour une tendance, 1000 pour une conclusion, 10 000 pour une décision de conception.
3. **Notez la graine et les réglages avec le chiffre.** Un winrate sans son contexte n'est pas une mesure, c'est une anecdote. La barre du plateau affiche la graine en permanence pour cette raison.
4. **Le résultat qui ne bouge pas est un résultat.** Si doubler la portée ne change pas le winrate, vous venez d'apprendre que ce paramètre n'est pas un levier. C'est une information, pas un échec.

Le réglage modifié en cours de partie

Le panneau *Règles* affiche un bandeau orange dès que vous touchez un paramètre : « Réglage modifié — relance la partie pour mesurer ». Il est là parce que les coups **déjà joués** ne sont pas rejoués avec la nouvelle règle. Une partie à cheval sur deux réglages ne mesure rien.

6.8 Quand ça ne marche pas

Symptôme	Regardez d'abord
rien ne bouge	la barre d'état du plateau — elle dit ce qu'on attend
mon module n'apparaît pas	panneau <i>Modules</i> : détecté ? compilé ? bon chemin ?
« Symbole introuvable »	les deux <code>NKC_MODULE_EXPORT</code> en fin de fichier
« ABI 2, attendue 3 »	vous compilez contre de vieux en-têtes
« coup ILLÉGAL »	le memset de votre coup
l'interface se fige une seconde	c'est une compilation, pas un plantage
« le moteur a REFUSÉ ce plateau »	cells vide, JSON malformé, ou moteur sans <code>LoadBoardJson</code>
la campagne refuse de démarrer	chaque siège doit être tenu par une IA <i>chargée</i>

6.9 Le banc d'essai, hors interface

Avant de croire à quoi que ce soit, faites tourner le banc d'essai. Il vérifie le contrat de bout en bout, sans interface :

Applications/ConquerorLab/README.md — section « Reproduire »

```
1 CLANG="C:/msys64/ucrt64/bin/clang++.exe"
2 R="d:/Projets/2026/Nkentseu/Nkentseu"
3 INC="-I$R/Applications/ConquerorLab/include \
4     -I$R/Kernel/Foundation/NKCore/src \
5     -I$R/Kernel/Foundation/NKPlatform/src"
6
7 $CLANG -shared -std=c++17 -O2 -fPIC $INC -o ConquerorRulesV2.dll \
8     $R/Applications/ConquerorLab/modules/rules/ConquerorRulesV2.cpp
9 $CLANG -shared -std=c++17 -O2 -fPIC $INC -o ConquerorAIRef.dll \
10    $R/Applications/ConquerorLab/modules/ai/ConquerorAIRef.cpp
11 $CLANG -std=c++17 -O1 $INC -o abitest.exe \
12    $R/Applications/ConquerorLab/tests/NkcAbiHarness.cpp
13 ./abitest.exe
```

Seize vérifications, dont les trois qui comptent : l'IA ne produit jamais de coup illégal, la partie se termine d'elle-même, et le jeu redonne exactement la même empreinte.

Remplacez les deux chemins de modules par les vôtres : le banc d'essai teste **votre** module aussi bien que celui de référence.

1 — Le premier chiffre

Lancez 200 parties, IA de référence *Normal* contre *Facile*, sur le plateau par défaut. Notez : winrate par camp, victoires du siège 1, taux de coupées, durée moyenne. Recommencez avec une autre graine. Les chiffres bougent-ils ? De combien ? Vous venez de mesurer votre bruit de fond — donc le seuil en dessous duquel un écart ne veut rien dire.

2 — L'avantage au premier joueur

Même IA des deux côtés, 1000 parties. Que vaut « Siège 1 gagne » ? Est-ce significatif au regard du bruit mesuré à l'exercice 1 ? C'est la question du palier 0 : écrivez votre réponse en une phrase, avec le chiffre.

3 — Le paramètre qui ne sert à rien

Prenez un paramètre du moteur de référence et faites-le varier sur trois valeurs, 300 parties chacune. Tracez le winrate. Est-ce un levier ou une décoration ? Sur quel critère tranchez-vous ?

4 — Reproduire une divergence

Introduisez volontairement du non-déterminisme dans votre moteur : par exemple, faites dépendre l'ordre des coups de l'adresse d'un pointeur. Jouez une partie, copiez le journal, relancez avec la même graine, comparez les empreintes. À quel coup la divergence apparaît-elle ? Retirez ensuite le défaut.

Exemples complets — l'échelle, barreau par barreau

Les chapitres précédents expliquaient le contrat. Celui-ci donne **du code qui tourne**, du plus court au plus complet, pour chacune des trois choses qu'on écrit : des règles, une IA, un plateau.

Ne sautez pas de barreau

Chaque exemple existe parce que le suivant serait incompréhensible sans lui. [IANegamax.cpp](#) fait cinq cents lignes; il en fait cent pour qui a d'abord lu [IAGloutonne.cpp](#), et mille pour qui ne l'a pas lu.

7.1 Les trois échelles

Écrire...	Le plus court	L'intermédiaire	Le complet
une IA	IAMinimale.cpp ≈120 l. — au hasard	IAFacile.cpp ≈140 l. — un coup	IANegamax.cpp ≈500 l. — négamax $\alpha\beta$
des règles	RegleContratNu.cpp carré 5×5	—	ConquerorRulesV2.cpp le moteur de référence
un plateau	mini_3x3.json 9 cases	hexagone_6x7.json 42 cases	GrilleLibre.cpp en C++, projection comprise

Tous sont dans `exemples/`, sauf le moteur de référence. Tous compilent, et les duels ci-dessous ont été joués, pas supposés.

7.2 D'abord : la couche confortable

Le contrat est fait pour être **stable** — structures plates, pointeurs bruts, aucune allocation. C'est ce qu'il faut pour une frontière binaire, et c'est pénible à lire.

Compter « les ennemis autour de mes totems » avec le contrat nu :

Le contrat nu — trois boucles imbriquées

```

1 NkcStateView v;
2 std::memset(&v, 0, sizeof(v));
3 rules->GetView(inst, st, &v);
4 for (uint32 i = 0; i < v.cellCount; ++i) {
5     if (v.cells[i].owner != (int8)moi) continue;
6     NkcCoord nb[16];
7     const uint32 n = rules->GetNeighbors(inst, v.coords[i], nb, 16);
8     for (uint32 k = 0; k < n; ++k)
9         for (uint32 j = 0; j < v.cellCount; ++j) // recherche a la main

```

```

10         if (v.coords[j].q == nb[k].q && v.coords[j].r == nb[k].r) {
11             if (v.cells[j].owner >= 0 &&
12                 v.cells[j].owner != (int8)moi) ++c;
13             break;
14         }
15     }

```

Avec `Conqueror/ConquerorFacile.h` :

La meme chose

```

1 Plateau p(*rules, inst, st);
2 for (Case c : p)
3     if (c.AMoi(moi))
4         contact += p.CompteVoisins(c.ou, Voisin::Ennemi, moi);

```

7.2.1 Ce qu'elle apporte

Classe	À quoi ça sert
Plateau	lire l'état : <code>QuiJoue()</code> , <code>Totems(j)</code> , <code>Avantage(moi)</code> , <code>A(coord)</code> , <code>Voisins()</code> , <code>CompteVoisins()</code> — et <code>for (Case c : p)</code>
Case	une case avec sa coordonnée : <code>Vide()</code> , <code>Bloquee()</code> , <code>AMoi(j)</code> , <code>Ennemie(j)</code>
Coups<N>	les coups légaux, itérables
Essai	jouer un coup pour de faux : <code>clone</code> , <code>applique</code> , <code>libère tout seul</code>

Rien de nouveau, et rien à désapprendre

Chaque fonction appelle le contrat, et rien d'autre. Tout ce qu'on fait avec ce fichier, on peut le faire sans — en plus long.
 Tout est **inline** et **sans allocation** : aucun symbole à lier, rien qu'un compilateur en -O2 ne supprime. Un `Plateau` tient en quelques pointeurs.

Un piège quand même

Créez l'`Essai` **hors** de la boucle des coups candidats. En créer un par candidat coûte une allocation d'état par candidat — et c'est exactement la boucle chaude d'une IA.

7.2.2 La preuve : la même IA, deux fois

`exemples/ai/IAFacile.cpp` est le **même algorithme** que `IAGloutonne.cpp`, écrit avec la couche : ≈ 140 lignes contre ≈ 240 .

Duels joués le 2026-08-09 — 40 parties, cotes inversees

```

1 IAFacile vs IAGloutonne 20 - 20 50 % <- comportement identique
2 IAFacile vs IAMinimale 40 - 0 100 %

```

Le 20–20 est le résultat **attendu** de deux joueurs identiques. C'est ce qui prouve que la couche ne change rien au jeu — seulement à ce qu'on écrit.

Par où commencer

Lisez `IAFacile.cpp` d'abord : sa partie qui *décide* tient en quinze lignes. Lisez `IAGloutonne.cpp` ensuite, pour voir ce que la couche vous épargne, et pour savoir quoi faire le jour où elle ne suffira plus.

7.3 Une IA, en trois temps

7.3.1 Temps 1 — tirer au hasard

Elle ne réfléchit pas. Son seul mérite est de montrer la **forme d'un module d'IA** : neuf fonctions, dont la plupart ne font rien.

exemples/ai/IAMinimale.cpp — le cœur

```
1  const uint32 total = rules->GenerateLegalMoves(inst, st, a->moves, kMaxMoves);
2  const uint32 n      = total < kMaxMoves ? total : kMaxMoves;
3  if (n == 0) return 0;           // aucun coup : L'atelier jouera PASSER
4  out->move = a->moves[Rand(a) % n];
5  return 1;
```

Retenez surtout ce qu'elle **ne fait pas** : elle ne construit pas de coup. Elle **demande** les coups légaux au moteur et en choisit un. Une IA qui fabrique ses propres coups produit tôt ou tard un coup illégal.

7.3.2 Temps 2 — regarder un coup

Le vrai premier pas. L'algorithme tient en quatre lignes :

L'algorithme glouton, en entier

```
1  pour chaque coup legal :
2      cloner l'etat, jouer le coup pour de faux
3      donner une note a la position obtenue
4  garder le coup dont la note est la meilleure
```

exemples/ai/IAGloutonne.cpp — jouer pour de faux

```
1  for (uint32 i = 0; i < n; ++i) {
2      // JOUER POUR DE FAUX : on clone, on applique, on note. L'etat reçu
3      // n'est JAMAIS modifié -- Le contrat l'interdit, et l'atelier compte
4      // dessus pour faire tourner plusieurs parties en parallèle.
5      rules->CloneState(inst, a->essai, st);
6      if (!rules->ApplyMove(inst, a->essai, &a->moves[i], nullptr, nullptr))
7          continue; // Le moteur a refusé : il a toujours le dernier mot
8
9      const int32 note = Noter(rules, inst, a->essai, moi, a->poidsTotems);
10
11     // `>` et non `>=` : a note égale on garde Le PREMIER. L'ordre des
12     // coups est fixe par Le moteur, donc ce choix est REPRODUCTIBLE.
13     if (premier || note > meilleurNote) {
14         meilleurNote = note; meilleur = i; premier = false;
15     }
16 }
```

L'évaluation est volontairement pauvre — un seul critère, le nombre de totems :

exemples/ai/IAGloutonne.cpp — Noter

```
1 int32 miens = 0, siens = 0;
2 for (uint8 p = 0; p < v.playerCount; ++p) {
3     if (p == moi) miens += v.totemCount[p];
4     else          siens += v.totemCount[p];
5 }
6 return (miens - siens) * poids;
```

Ce qu'elle ne sait pas faire, et c'est le point

Elle ne voit pas la **réponse** de l'adversaire. Un coup qui gagne trois totems et en offre cinq au coup suivant lui paraît excellent.

C'est exactement le manque que négamax comble. Le meilleur moyen de comprendre pourquoi négamax existe est d'avoir d'abord vu jouer celle-ci.

7.3.3 Temps 3 — regarder loin

Quatre pièces, dans l'ordre où elles comptent : **évaluer, négamax, alpha-bêta, approfondissement itératif**.

Pourquoi négamax et pas minimax

Minimax écrit deux fonctions, une qui maximise et une qui minimise. Négamax n'en écrit qu'une, en observant que $\min(a, b) = -\max(-a, -b)$.

La valeur d'une position pour le joueur au trait est l'opposé de sa valeur pour l'autre. On **évalue toujours du point de vue du joueur au trait**.

exemples/ai/IANegamax.cpp — la boucle centrale

```
1 // L'enfant rend SA valeur, du point de vue de SON joueur au trait. On
2 // la ramène au point de vue de `mover` : identique si c'est le même
3 // joueur qui rejoue (coup gratuit, tour multiple), opposée sinon.
4 if (after.current == mover) {
5     score = Negamax(ai, rules, inst, ai->work[ply], depth - 1,
6                     alpha, beta, ply + 1);
7 } else {
8     score = -Negamax(ai, rules, inst, ai->work[ply], depth - 1,
9                     -beta, -alpha, ply + 1);
10 }
11
12 if (ai->outOfTime) return 0;
13 if (score > best) best = score;
14 if (best > alpha) alpha = best;
15 if (alpha >= beta) break; // COUPURE BETA
```

L'erreur de signe — elle s'est produite ici, en écrivant ce fichier

La première version évaluait toujours du point de vue du joueur **racine**, tout en négligeant à chaque changement de trait. Les deux conventions se contredisaient.

Rien ne plantait. Aucun test ne tombait. L'IA cherchait simplement, très efficacement, le pire coup — et perdait 16 à 20 contre un joueur aléatoire.

Le remède n'a pas été de corriger un signe, mais de **supprimer la possibilité de l'erreur** : le paramètre `me` a disparu de `Negamax`. Tant qu'une fonction porte à la fois «le joueur racine» et «le joueur au trait», il existe un endroit où l'on peut confondre les deux, et il

suffit d'un.

Alpha-bêta

α est le meilleur score que le joueur au trait s'est déjà garanti ; β celui que l'adversaire s'est garanti ailleurs. Dès que $\text{score} \geq \beta$, on arrête : l'adversaire ne laissera jamais la partie arriver ici.

Le résultat est **exactement** celui du négamax nu ; seul le nombre de nœuds change. Et cette économie dépend entièrement de l'**ordre** des coups — d'où le tri, qui est un paramètre pour qu'on puisse mesurer ce qu'il rapporte.

Approfondissement itératif

On cherche à profondeur 1, on garde le meilleur coup. Puis 2. Puis 3.

exemples/ai/IANegamax.cpp — Search

```
1 // LE POINT QUI FAIT TOUT : on ne garde le resultat d'un niveau que
2 // s'il a ete PARCOURU EN ENTIER. Un niveau interrompu a examine les
3 // premiers coups seulement, et le « meilleur » qu'il propose est le
4 // meilleur d'un echantillon arbitraire.
5 if (ai->outOfTime) break;
6 best      = levelBestMove;
7 bestScore = levelBest;
8 reached   = depth;
```

Cela paraît du gaspillage — on refait le travail à chaque fois. Ce n'en est pas : le dernier niveau coûte plus que tous les précédents réunis. Et c'est la **seule façon de respecter un budget de temps** : sans lui, il faudrait deviner la profondeur atteignable, et se tromper signifie soit figer l'interface, soit ne rien chercher.

La limite assumée

IANegamax détecte les parties à plus de deux joueurs et le dit. L'identité $\min(a, b) = -\max(-a, -b)$ suppose un jeu à somme nulle à deux. À trois, « le gain de l'un est la perte de l'autre » est faux : il y a deux autres.

7.3.4 Ce que ça donne, mesuré

Banc de duel, 40 parties, côtés inversés une sur deux, budget 60 ms par coup, moteur ConquerorRulesV2 :

Duels joués le 2026-08-08

1	Gloutonne	vs	IAMinimale (aleatoire)	40 - 0	100 %
2	Negamax	vs	Gloutonne	40 - 0	100 %
3	Negamax	vs	IAMinimale	40 - 0	100 %
4					
5	contrôles :				
6	Gloutonne	vs	Gloutonne	20 - 20	50 %
7	IAMinimale	vs	IAMinimale	20 - 20	50 %

Pourquoi les deux dernières lignes comptent plus que les trois premières

Un banc qui donne toujours 40–0 est un banc cassé. Les contrôles à 20–20 prouvent qu’il n’y a **ni biais de siège, ni bug de comptage** — et que les 100 % au-dessus mesurent bien les IA.

C’est le premier réflexe à prendre : avant de croire un chiffre, faire tourner le cas où l’on connaît déjà la réponse.

7.4 Des règles : trois fonctions, ou vingt et une

NkcRulesVTable compte **vingt et une** fonctions. Sur ces vingt et une, **trois** contiennent votre jeu :

Fonction	Ce qu’elle dit
Construire	à quoi la partie ressemble au départ
CoupsPossibles	ce qu’on a le droit de faire
Appliquer	ce qui se passe quand on le fait

Les dix-huit autres sont les mêmes pour tout le monde. [ConquerorRegleFacile.h](#) les écrit une fois : [RegleContratNu.cpp](#) fait ≈ 560 lignes, [RegleFacile.cpp](#) en fait ≈ 110 .

exemples/rules/RegleFacile.cpp — le jeu entier

```
1 struct RegleFacile {
2     void Construire(Grille &g) {
3         g.topologie = NkcTopology::Square4;
4         g.nbJoueurs = 2;
5         g.AjouterRectangle(5, 5);
6         g.PoserAuxCoins(2); // symetrie 4.2
7     }
8
9     void CoupsPossibles(const Partie &p, ListeCoups &out) {
10        NkcCoord voisins[8];
11        for (int32 i = 0; i < p.nbCases; ++i) {
12            if (p.cases[i].owner != (int8)p.joueur) continue;
13            const int32 n = p.Voisins(p.ou[i], voisins, 8);
14            for (int32 k = 0; k < n; ++k)
15                if (p.Vide(voisins[k]))
16                    out.Dupliquer(p.joueur, p.ou[i], voisins[k]);
17        }
18    }
19
20    void Appliquer(Partie &p, const NkcMove &m, Evenements &ev) { /* ... */ }
21 };
22 NKC_REGLES(RegleFacile, "RegleFacile", "1.0.0", "Cours ConquerorLab")
```

7.4.1 Pourquoi les dix-huit autres deviennent gratuites

Parce que **l’état cesse d’être libre**. Le contrat dit «le module choisit sa représentation interne» — c’est sa force, et c’est ce qui coûte cher : tant que la structure est inconnue, personne ne peut écrire sa sérialisation ni son empreinte à votre place.

Partie est une structure **fixe, plate, sans pointeur**. Du coup :

Fonction	Devient
SerializeState	un memcpy
CloneState	une affectation
HashState	FNV-1a sur les octets — identique sur toute plateforme
IsLegalMove	énumérer et comparer

Et ces quatre-là sont correctes **par construction**, pas par relecture.

Deux pièges classiques disparaissent

ListeCoups::Dupliquer fait le memset du coup avant de le remplir — l’erreur numéro un des premiers modules, puisque le contrat compare les coups **octet par octet**. Et PASSER est ajouté automatiquement quand la liste reste vide, jamais autrement : c’est exactement ce qu’exige REGLES §13.

Ce qu’on perd, et quand il faudra partir

Le jour où votre moteur a besoin d’un état que Partie ne sait pas porter — une pile de pouvoirs, un historique, un cache d’évaluation — l’échafaudage ne suffit plus. Vous reprendrez le contrat nu, et vous saurez exactement quelles dix-huit fonctions écrire, **parce que vous les aurez vues à l’œuvre**.

C’est un échafaudage, pas une cage. Rien n’est caché : tout est inline et lisible dans le header.

7.4.2 Ce que le banc d’essai en dit

RegleFacile sous tests/NkcAbiHarness.cpp

```
1 === 13 verifications OK, 3 echecs ===
```

Les trois échecs sont ceux **codés en dur sur le moteur de référence** : le banc cherche portee_duplication, «42 cases» et «2 totems par joueur». RegleFacile joue un 5×5 avec un totem chacun. Même profil que RegleContratNu (14/16) et GrilleLibre (13/16).

Tout ce qui porte sur le **contrat** passe — y compris le rejeu déterministe, le bornage des paramètres et le garde-fou max_tours.

Une correction que ce banc a provoquée

À la première version, l’échafaudage n’exposait **aucun paramètre** et n’avait **aucun garde-fou** : un jeu de règles pathologique pouvait boucler indéfiniment, et la campagne ne rendait jamais la main. Le banc l’a signalé en deux lignes. max_tours est désormais fourni d’office, et il n’est pas optionnel.

7.5 Des règles, sans échafaudage : RegleContratNu.cpp

Vingt et une fonctions dans la vtable, dont la moitié tient en trois lignes. Le chapitre 2 les parcourt une à une ; voici seulement ce qui structure le fichier.

exemples/rules/RegleContratNu.cpp — le plateau, en C++

```
1 for (int32 y = 0; y < kSide; ++y)
2     for (int32 x = 0; x < kSide; ++x) {
3         R->coords[y * kSide + x].q = static_cast<int16>(x);
4         R->coords[y * kSide + x].r = static_cast<int16>(y);
5     }
```

Le memset sur chaque coup

```
NkcMove m; std::memset(&m, 0, sizeof(m)); // comparaison octet a octet
```

Le contrat compare les coups **octet par octet**. Un champ non initialisé — même inutilisé — rend le coup différent de lui-même, et `IsLegalMove` le refuse. C'est l'erreur numéro un des premiers modules.

Le moteur complet, `modules/rules/ConquerorRulesV2.cpp`, ajoute le chargement de plateau JSON, les paramètres, la sérialisation et l'empreinte. Lisez-le **après** avoir écrit le vôtre, pas avant.

7.6 Un plateau : les deux voies

7.6.1 En JSON, le plus petit possible

exemples/boards/mini_3x3.json

```
1 {
2     "topology": "SQUARE_4",
3     "cells": [[0,0],[1,0],[2,0],
4               [0,1],[1,1],[2,1],
5               [0,2],[1,2],[2,2]],
6     "blocked": [[1,1]],
7     "starts": [
8         {"player": 0, "q": 0, "r": 0, "level": 0},
9         {"player": 1, "q": 2, "r": 2, "level": 0}
10    ],
11     "min_players": 2, "max_players": 2
12 }
```

Neuf cases, une bloquée au centre, deux totems opposés. Une partie dure quinze secondes et on suit chaque coup à l'œil.

C'est le plateau sur lequel on débogue

Chercher pourquoi un moteur se trompe sur 42 cases est un travail d'archéologue. Sur neuf, on voit.

Réflexe à prendre : dès qu'un comportement paraît étrange, **rechargez mini_3x3.json** avant de lire une ligne de code.

7.6.2 En C++, quand la forme est une formule

`exemples/rules/GrilleLibre.cpp` — trois anneaux concentriques, 19 cellules. Ni hexagone, ni carré. Le chapitre 5 le détaille; reprenez les deux entrées qui rendent la chose possible, et celle qui rend la grille **utilisable par une IA** :

Les trois entrees de geometrie (ABI 3)

```
1 int32 (*GetCellCenter)(NkcRules self, NkcCoord c, float32 *outXY);
2 uint32 (*GetCellShape) (NkcRules self, NkcCoord c, float32 *outXY, uint32 cap);
3 uint32 (*GetNeighbors) (NkcRules self, NkcCoord c, NkcCoord *out, uint32 cap);
```

Sans `GetNeighbors`, une IA devine l'adjacence à partir de `topology` — et devine faux, en silence. `IANegamax` s'en sert pour son terme de menace, et **s'abstient** de ce terme quand elle n'est pas déclarée : une évaluation qui devine faux est pire qu'une évaluation qui ignore le critère.

7.7 Le chemin conseillé

1. Copiez `IAMinimale.cpp` dans `travail/ai/`, changez son nom dans `FillFactory`, sauvegardez. Vérifiez qu'elle apparaît dans le panneau *Joueurs*.
2. Lisez `IAFacile.cpp` — quinze lignes décident, le reste est de la plomberie qu'on recopie.
3. Écrivez votre évaluation : partez d'`IAFacile.cpp`, changez `Noter`, et rien d'autre. C'est là qu'est le vrai travail d'une IA de jeu.
4. Faites-la jouer contre l'aléatoire, 40 parties. Vous devez gagner largement. Sinon, c'est un signe — pas une malchance.
5. Puis seulement, ouvrez `IANegamax.cpp`.

1 — Le contrôle avant la mesure

Faites jouer `IAGloutonne` contre elle-même, 100 parties. Attendu : environ 50 %. Si vous obtenez autre chose, cherchez pourquoi **avant** toute autre mesure — vous venez de trouver un biais de siège ou un bug de comptage.

2 — Casser le signe, exprès

Dans `IANegamax.cpp`, remplacez `-Negamax(...)` par `Negamax(...)` dans la branche du changement de joueur. Recompilez, faites-la jouer contre l'aléatoire. Que se passe-t-il ? Combien de temps auriez-vous mis à trouver sans connaître la réponse ?

3 — Ce que rapporte le tri

`IANegamax` expose `tri_des_coups`. Coupez-le, relancez une campagne à profondeur fixe, comparez le nombre de nœuds. De combien l'alpha-bêta est-il moins efficace sans tri ? Le chiffre vous appartient — le facteur théorique est connu, mais ce jeu-ci n'est pas la théorie.

4 — Un troisième critère

Ajoutez à `IAGloutonne` un terme de mobilité (nombre de coups disponibles) et mesurez. Le gain est-il réel, ou dans le bruit ? Rappel du chapitre 6 : 40 parties ne suffisent pas à trancher un écart de cinq points.

5 — Le budget

Passez le budget de `IANegamax` de 60 ms à 5 ms. Quelle profondeur atteint-elle encore ? Gagne-t-elle toujours contre l'aléatoire ? Vous mesurez là le rapport entre **temps de ré-**

flexion et force, qui est la seule chose que règle vraiment un palier de difficulté.

Quand l'échafaudage ne suffit plus : le contrat nu

Vous n'avez pas besoin de ce chapitre pour écrire un jeu. Trois fonctions suffisent, et c'était le chapitre 2. Celui-ci est pour le jour où votre état de partie ne rentre plus dans la structure `Partie` — une liste qui grandit, un cache, un arbre. Ce jour-là, vous écrivez les vingt-quatre entrées vous-même, et tout ce qui suit vous servira.

Il se lira d'ailleurs beaucoup mieux maintenant que vous avez vu les dix-huit fonctions à l'œuvre.

Ce chapitre construit un moteur complet, de la première ligne à la dernière. Le fichier existe, il compile, et l'atelier le charge : `Applications/ConquerorLab/exemples/rules/RegleContratNu.cpp`.

Il joue **exactement le même jeu** que `exemples/rules/RegleFacile.cpp` du chapitre 2 : 570 lignes au lieu de 110, et pas une règle de plus. Comparer les deux est l'exercice le plus instructif du cours.

Ouvrez-le à côté de ce texte. Nous le parcourons dans l'ordre, en expliquant à chaque étape *pourquoi* elle est là et *ce qui casse* si on l'oublie.

8.1 Ce que ce moteur fait

RegleContratNu.cpp — resume

```
1 plateau      carre 5x5, 4 voisins (pas d'hexagone : une chose a la fois)
2 action       DUPLIQUER uniquement, portee 1
3 transformation les ennemis voisins de la case OU L'ON VIENT DE POSER
4              changent de camp -- et eux seuls
5 fin          des qu'un joueur ne peut plus dupliquer
6 decompte     le plus de totems gagne
```

C'est déjà un jeu jouable, et c'est assez pour que l'atelier fasse **tout** ce qu'il sait faire : partie humaine, IA, journal, rejou, campagne de mesure.

8.2 Les vingt-trois fonctions, par famille

`NkcRulesVTable` compte vingt-trois entrées depuis l'ABI 3. Elles se rangent en cinq familles, et cette carte suffit à ne jamais se perdre.

Famille	Fonctions	Rôle
cycle de vie	Create, Destroy, CreateState, DestroyState, CloneState, Setup	fabriquer un moteur, puis des parties
réglages	GetParamsSchemaJson, SetParam, GetParam, LoadBoardJson, GetBoardJson	ce qui change sans recompiler
jeu	GetView, GenerateLegalMoves, IsLegalMove, ApplyMove, IsFinished, GetWinner, IsPlayerBlocked	les règles proprement dites
sérialisation	SerializeState, DeserializeState, HashState	rejeu, thread d'IA, diagnostic
géométrie (ABI 3)	GetCellCenter, GetCellShape	optionnel — chapitre 5

Ce qu'il faut retenir

Convention de retour, partout : **0 = échec**, **1 = succès**. Une fonction qui échoue ne doit **rien** avoir modifié.

8.3 La mémoire : jamais de new ni de delete bruts

exemples/rules/RegleContratNu.cpp — section 1

```

1 NkcAllocFn gAlloc = nullptr;
2 NkcFreeFn gFree = nullptr;
3
4 void *RawAlloc(usize n) { return gAlloc ? gAlloc(n) : std::malloc(n); }
5 void RawFree(void *p) { if (!p) return; if (gFree) gFree(p); else std::free(p); }
```

L'atelier peut vous injecter ses propres allocateurs (NKMmemory) par `nkc_rules_set_allocator`. Les deux chemins doivent exister : avec injection, et sans.

Ne jamais mélanger deux tas

C'est une règle générale du dépôt : allouer avec l'allocateur du moteur et libérer avec `std::free` (ou l'inverse) corrompt le tas — sous Windows, cela donne un `c0000374` sans pile utile, souvent très loin du vrai coupable. Le couple `RawAlloc/RawFree` garantit la symétrie.

8.4 Les paramètres : aucune constante en dur

exemples/rules/RegleContratNu.cpp — section 2

```

1 enum ParamId : int32 {
2     P_PORTEES = 0, // distance maximale de duplication
3     P_MAX_TOURS, // garde-fou de simulation, jamais une regle de jeu
4     P_COUNT
5 };
6
7 struct ParamDesc { const char *key, *group; int32 def, lo, hi; };
8
9 const ParamDesc kParams[P_COUNT] = {
```



```

10     {"portee_duplication", "Duplication", 1, 1, 3},
11     {"max_tours",          "Fin de partie", 200, 10, 100000},
12 };

```

Une seule table décrit tout : la clé, le groupe d’affichage, la valeur par défaut, les bornes. **Ajouter un réglage = ajouter une ligne.** Voici pourquoi cela suffit à faire apparaître un champ à l’écran :

exemples/rules/RegleContratNu.cpp — V_GetParamsSchemaJson

```

1 k = std::snprintf(w, left,
2     "%s{\"key\":\"%s\", \"group\":\"%s\", \"type\":\"int\", \"
3     \"min\":%d, \"max\":%d, \"def\":%d, \"val\":%d}\",
4     i ? ", \" : \"\", kParams[i].key, kParams[i].group,
5     kParams[i].lo, kParams[i].hi, kParams[i].def, R->params[i]);

```

Le panneau *Règles* de l’atelier est **entièrement auto-généré** à partir de cette chaîne. Il n’y a pas une ligne d’interface par paramètre.

"type"	Widget	Champs supplémentaires
"int"	champ numérique	min, max
"bool"	case à cocher	—
"enum"	liste déroulante	"values":["A", "B", ...]

Deux champs optionnels partout : "label" (sinon dérivé de la clé) et "help" (affiché en infobulle).

8.4.1 Borner, ne pas refuser

exemples/rules/RegleContratNu.cpp — V_SetParam

```

1 int32 v = static_cast<int32>(value < 0 ? value - 0.5 : value + 0.5);
2 if (v < kParams[i].lo) v = kParams[i].lo;
3 if (v > kParams[i].hi) v = kParams[i].hi;
4 R->params[i] = v;
5 return 1;

```

Le banc d’essai vérifie ce comportement

tests/NkcAbiHarness.cpp:121-124

```

1 // Hors plage : doit etre borne au minimum du parametre (10), pas accepte.
2 R.SetParam(ri, "max_tours", 3);
3 CHECK(R.GetParam(ri, "max_tours") == 10.0,
4     "une valeur hors plage est bornee, pas acceptee telle quelle");

```

Le panneau *Règles* relit le schéma à **chaque image** et n’a aucun cache : si vous bornez, l’utilisateur voit tout de suite la valeur bornée. Un cache local côté interface mentirait.

Notez que SetParam prend un float64 — c’est le **seul** flottant de la logique du contrat, et il ne la traverse pas : c’est un canal de réglage, converti en entier dès la première ligne.

8.5 L'état : votre représentation, leur vue

exemples/rules/RegleContratNu.cpp — section 4

```
1 struct State {
2     NkcCellView cells[kCells];
3     uint8      playerCount = 2;
4     uint8      current      = 0;
5     uint8      finished     = 0;
6     int8       winner       = -2;
7     uint32     turn         = 0;
8     int32      energy[kMaxPlayers] = {};
9     int32      conquestTenths[kMaxPlayers] = {};
10    int32      totemCount[kMaxPlayers] = {};
11    uint64     rng = 0x9E3779B97F4A7C15ull; // PRNG PORTE PAR L'ETAT
12 };
```

Le contrat n'impose rien de cette structure. Une seule contrainte, mais elle est dure.

L'état doit se copier tel quel

CloneState est le **chemin chaud de l'IA** : appelé des milliers de fois par seconde. Le contrat dit : « doit être rapide et ne jamais allouer ». D'où :

RegleContratNu.cpp — V_CloneState

```
1 void V_CloneState(NkcRules, NkcState dst, const NkcState src) {
2     if (dst && src) *static_cast<State *>(dst) = *static_cast<const State *>(src);
3 }
```

Une seule affectation, parce que State est un POD. Si vous y mettez un conteneur qui alloue, cette ligne alloue aussi, et votre IA perd un ordre de grandeur.

La même contrainte explique SerializeState :

RegleContratNu.cpp — V_SerializeState

```
1 uint32 V_SerializeState(NkcRules, const NkcState st, void *buf, uint32 cap) {
2     const uint32 need = static_cast<uint32>(sizeof(State));
3     if (!buf || cap < need) return need; // renvoie la taille NECESSAIRE
4     std::memcpy(buf, st, need);
5     return need;
6 }
```

Le protocole en deux temps

Appelée avec `buf == nullptr`, la fonction doit renvoyer la **taille nécessaire** sans rien écrire. C'est ainsi que l'atelier dimensionne son tampon avant de transférer l'état vers le thread d'IA. Renvoyer 0 dans ce cas fait échouer la réflexion — et l'atelier affichera « SerializeState annonce une taille nulle » dans la barre du plateau.

8.6 Générer les coups

C'est la fonction la plus importante du fichier. Trois exigences s'y croisent.

```

1  for (int32 i = 0; i < kCells; ++i) {
2      if (s->cells[i].owner != static_cast<int8>(me)) continue;
3      const NkcCoord src = R->coords[i];
4
5      for (int32 k = 0; k < nbr; ++k) {
6          const NkcCoord dst = Neighbor(NkcTopology::Square4, src, k);
7          const int32 di = IndexOf(dst);
8          if (di < 0) continue;
9          if (s->cells[di].owner != kCellEmpty) continue;
10         if (Distance(NkcTopology::Square4, src, dst) > range) continue;
11
12         if (n < cap) {
13             NkcMove m;
14             std::memset(&m, 0, sizeof(m));
15             m.kind = NkcMoveKind::Duplicate;
16             m.player = me;
17             m.from = src;
18             m.to = dst;
19             m.targetLevel = -1;
20             m.powerId = -1;
21             out[n] = m;
22         }
23         ++n;
24     }
25 }

```

Exigence 1 — l'ordre est normatif. Cas par index croissant, puis voisins dans l'ordre de Neighbor. Deux exécutions produisent la même liste, sur toute plateforme.

Exigence 2 — la mise à zéro est obligatoire.

memset avant de remplir, toujours

Coups. POD comparable octet à octet APRÈS mise à zéro : le module DOIT remplir intégralement les champs inutilisés avec zéro. CONQUERORRULESABI.H :112

NkcMove contient NkcCoord fuseCells[12], inutilisés au palier 0. Sans memset, ils contiennent ce que la pile y a laissé. Or IsLegalMove compare les coups **octet à octet** : deux coups identiques deviendraient différents, et l'atelier rejetterait le coup de votre propre IA. Le symptôme est spectaculaire — « l'IA a proposé un coup ILLÉGAL » — et la cause est trois lignes plus haut.

Exigence 3 — cap peut être trop petit. La fonction écrit *au plus* cap coups mais renvoie le nombre *total*. Remarquez que ++n est en dehors du if (n < cap) : l'appelant apprend ainsi qu'il doit agrandir son tampon.

8.6.1 Le cas de PASSER

```

1  // PASSER n'est Legal que si RIEN d'autre ne l'est (REGLES 6.2).
2  if (n == 0) {
3      if (cap > 0) { /* ... un coup Pass ... */ }
4      return 1;
5  }

```

Ce n'est pas un détail de confort : c'est ce qui rend vraie l'équivalence suivante, que le banc d'essai vérifie automatiquement.

8.7 L'équivalence qui teste tout le reste

tests/NkcAbiHarness.cpp:107-111

```
1 const uint32 n = R.GenerateLegalMoves(ri, probe, buf, 512);
2 const bool onlyPass = (n == 1 && buf[0].kind == NkcMoveKind::Pass);
3 const bool blocked = R.IsPlayerBlocked(ri, probe, pv.current) != 0;
4 if (onlyPass != blocked) { equiv = false; break; }
```

Autrement dit : «aucun coup légal» doit être exactement équivalent à «joueur bloqué». Les règles le disent en toutes lettres :

Aucune action légale mais joueur non bloqué : impossible par construction. Le moteur doit **asserter cette équivalence en test** : c'est le meilleur test d'intégrité du générateur de coups.
REGLES_COMPLETES_V2.MD :467

Pourquoi ce test attrape presque tout

GenerateLegalMoves et IsPlayerBlocked sont deux chemins de code indépendants qui répondent à la même question. S'ils divergent, l'un des deux a tort — et c'est presque toujours le générateur, sur un cas de bord : case hors plateau, case bloquée, portée mal comparée.

Dans [RegleContratNu.cpp](#) les deux partagent délibérément la même primitive, CanPlay, ce qui rend l'équivalence vraie par construction.

8.8 Appliquer un coup, et la cascade

exemples/rules/RegleContratNu.cpp — DoApply, placement

```
1 // --- placement : la source RESTE INTACTE (REGLES 7.2) ---
2 s->cells[di].owner = static_cast<int8>(mv->player);
3 s->cells[di].level = 0;
4 Emit(sink, user, NkcEventKind::TotemDuplicated, mv->player, mv->from, mv->to, 0);
5 s->conquestTenths[mv->player] += 10; // +1,0 PC, en DIXIEMES entiers
```

Puis la transformation, et c'est ici qu'on se trompe :

exemples/rules/RegleContratNu.cpp — DoApply, transformation

```
1 const int32 nbr = NeighborCount(NkcTopology::Square4);
2 int32 flipped = 0;
3 for (int32 k = 0; k < nbr; ++k) {
4     const int32 ni = IndexOf(Neighbor(NkcTopology::Square4, mv->to, k));
5     if (ni < 0) continue;
6     const int8 o = s->cells[ni].owner;
7     if (o < 0 || o == static_cast<int8>(mv->player)) continue;
8     /* ... retourner ce totem ... */
9     ++flipped;
10 }
11 if (flipped >= 2)
12     Emit(sink, user, NkcEventKind::Cascade, mv->player, mv->to, mv->to, flipped);
```

Une transformation ne déclenche pas de transformation

Une transformation ne déclenche pas de nouvelle transformation. Seul le totem *placé* est un déclencheur. Les cascades naissent de la multiplicité des voisins, pas d'une propagation. C'est le point le plus facile à implémenter de travers.
REGLES_COMPLETES_V2.MD :246

La boucle ci-dessus parcourt **une seule fois** les voisins de *mv->to*. Si vous la rappelez sur chaque case retournée, tout le plateau bascule au premier coup et le jeu n'existe plus. La « cascade » que l'atelier affiche en gros au centre de l'écran, c'est *flipped >= 2* — plusieurs voisins d'un coup, pas une réaction en chaîne.

8.8.1 Les événements : décrire, pas animer

Emit pousse un *NkcEvent* vers un collecteur fourni par l'appelant. Le moteur ne sait rien de l'animation : il décrit ce qui s'est produit, la présentation décide comment le montrer.

Ce collecteur peut être *nullptr* — et il l'est presque toujours, parce qu'une IA ne veut pas payer le coût des événements pendant ses simulations. **Testez-le** : Emit commence par `if (!sink) return;`

Les événements servent aussi à l'aperçu

Quand vous survolez une destination dans l'atelier, les totems qui seraient retournés s'en-tourent de rouge. L'atelier ne le *déduit* pas des règles : il clone l'état, joue le coup pour de faux avec un collecteur, et lit les *TotemTransformed*.

Conséquence : **votre aperçu reste juste même quand vous changez la règle de transformation**, sans qu'on touche à l'interface.

8.9 Fin de partie et garde-fou

exemples/rules/RegleContratNu.cpp — CheckEnd

```
1 // Garde-fou de SIMULATION : sans lui, une partie sur dix mille ne finit
2 // jamais et le batch tourne pour toujours (REGLES 12.3).
3 if (!over && static_cast<int32>(s->turn) >=
4     R->params[P_MAX_TOURS] * static_cast<int32>(s->playerCount))
5     over = true;
```

max_tours n'est PAS une règle de jeu

max_tours n'est pas une règle de jeu : c'est une sécurité pour que 10 000 parties IA-vs-IA se terminent toujours. Le rapport de batch doit indiquer combien de parties ont fini par ce garde-fou. **Un taux non nul signale une pathologie des règles, pas un réglage à monter.**
REGLES_COMPLETES_V2.MD :451

Le panneau *Métriques* affiche ce taux dans une tuile à part, rouge dès qu'il dépasse zéro. Si vous le voyez monter, ne montez pas *max_tours* : cherchez pourquoi vos parties ne se terminent pas.

8.10 Les deux symboles exportés

exemples/rules/RegleContratNu.cpp — fin de fichier

```
1 NKC_MODULE_EXPORT void nkc_rules_set_allocator(NkcAllocFn a, NkcFreeFn f)
2 { gAlloc = a; gFree = f; }
3
4 NKC_MODULE_EXPORT void nkc_rules_get_factory(NkcRulesFactory *out)
5 { FillFactory(out); }
```

L'erreur du premier jour

Les oublier donne, dans le panneau *Modules* :

src/ConquerorLab/NkcModuleHost.h — BuildAndLoad

```
1 e.log = "Symbole introuvable : ";
2 e.log += symFactory;
3 e.log += " - as-tu bien termine ton fichier par la macro d'export ?";
```

Le message a été écrit pour cette erreur précise, parce qu'elle arrive à tout le monde une fois.

Et la version d'ABI, dans `FillFactory` : `out->info.abiVersion = kRulesAbiVersion`; . L'atelier **refuse** un module dont l'`abiVersion` diffère, avec un message qui donne les deux numéros. Si vous lisez « ABI 2, attendue 3 », vous avez compilé contre de vieux en-têtes.

8.11 Votre premier module, en quatre gestes

1. copiez `exemples/rules/RegleContratNu.cpp` vers `Build/ConquerorLab/rules/mes\_regles.cpp`;
2. changez le nom dans `FillFactory` — sinon vous aurez deux entrées identiques dans le menu et vous ne saurez pas laquelle vous testez;
3. sauvegardez, attendez une seconde : le panneau *Modules* affiche votre module;
4. sélectionnez-le, puis **Nouvelle partie**.

À partir de là, modifiez **une chose à la fois** et mesurez. C'est tout le métier.

1 — La portée

`portee_duplication` est réglable de 1 à 3, mais `GenMoves` n'énumère que les *voisins immédiats* : passer la portée à 2 ne change donc rien. Corrigez-le. Indice : `Neighbor` ne donne que la distance 1 ; il vous faut parcourir les cases et filtrer par `Distance(...) <= range`.

2 — Casser l'équivalence exprès

Dans `V_IsPlayerBlocked`, remplacez `CanPlay(R, s, player)` par `true`. Recompilez, lancez une partie IA contre IA. Que se passe-t-il ? Reproduisez ensuite le test du banc d'essai à la main. Vous venez d'écrire votre premier test d'intégrité.

3 — Le memset manquant

Retirez le `std::memset(&m, 0, sizeof(m))` de `GenMoves`, recompilez, et faites jouer une IA. Décrivez ce que vous observez et **expliquez-le** par le fonctionnement de `IsLegalMove`. Remettez ensuite la ligne.

4 — Le troisième joueur

`V_Setup` force `playerCount` à 2. Faites-en un moteur à trois joueurs : placez un troisième totem de départ, faites tourner l'ordre de jeu, et vérifiez le décompte en cas d'égalité. Que doit-il se passer quand un joueur n'a plus aucun totem ?